

Using Bayesian Neural Networks for Active Learning

Kristian Severin Mengel-Niemann

Student reg. number: 201904957

201904957@post.au.dk / kmn1998@hotmail.com

MSc Cognitive Science — School of Communication and Culture — Aarhus University

Master's Thesis

Supervisor: Riccardo Fusaroli

GitHub Repository: https://github.com/kristianseverin/BNNs-for-Active-Learning.git

03/06/2024

Acknowledgements

I owe thanks to multiple people for helping me see this project through. First and foremost I need to express my gratitude towards my supervisor, Riccardo Fusaroli. Thank you for your valuable inputs and guidance. Moreover, I wish to thank Pekka Parviainen who met with me and introduced me to the topic of Bayesian Neural Networks. Furthermore, I would like to thank Stine Steen Høgenhaug and the people at EPICLE for allowing me to use their data for my thesis. A special thank you also goes to my girlfriend, Eleonore for being a motivator and constant source of support. Lastly, I need to thank Liv Dysthe Sønderland for drawing the image on the titlepage of this thesis.

Kristian Severin Mengel-Niemann
03/06/2024

Abstract

Traditional deep learning models do not supply uncertainty estimates for their predictions which poses potential problems when implementing such models in real-world contexts (Gawlikowski et al., 2022). Bayesian Neural Networks provide a way to integrate the Bayesian school of probabilistic thinking with the capabilities of traditional neural networks. In this thesis Bayesian Neural Networks are introduced, implemented, and tested in numerous different contexts, to try and answer what the the upsides, shortcomings and use-cases of Bayesian neural networks are, and how Bayesian neural networks could be used to alleviate data labelling costs involved in expert-domain dataset acquisition. Thus, a mathematical introduction is given to some of the most common implementations and inner workings of Bayesian neural networks in chapter 1. This provides background knowledge required to understand the implementation of Bayesian neural networks and is the point of departure for the models that are implemented in later chapters.

Sparse research has experimented with Bayesian neural network performance on non-curated benchmark datasets. Chapter 2 implements closed form Bayesian neural network layers derived from variational dropout and Bayes by Backprop models on both regression and classification tasks. The models are implemented on a synthesized dataset that is reflective of human grocery shopping behavior to showcase the models' predictive performances on a dataset that emulates human cognitive behavior. The models are trained on large datasets and all implemented models are tested on both a noisy and a relaxed version of the data to show how noise in the training data influences the models' predictive performances on never seen before data. It is shown that model architecture and type influence the predictive performance to a small extent for both regression and classification tasks, while noise in the data has a massive impact on the models' predictive abilities.

The uncertainty that Bayesian neural networks provide has been proposed to be used for acquiring the most informative unlabeled datapoints for annotation in active learning Rakesh Jain, (2021). Chapter 3 offers insight into what the consequences of training Bayesian neural networks on fewer annotated datapoints in rounds of active learning are. Thus, the relaxed datasets from chapter 2 are used with a tenth of the sample size to train regression and classification models in rounds of active learning. An oracle with noise inducing properties is queried with datapoints for annotation for each round of active learning. It is shown that Bayesian neural networks are robust to noise induced by imperfect oracles in active learning. Moreover, the models' predictive

performances on never seen before data are discussed with a comparison of results from chapter 2. It is concluded that Bayesian neural networks are highly practical for active learning pipelines regardless of model architecture and irrespective of the nature of the task.

Chapter 4 shows proof-of-concept for the use of Bayesian neural networks on real-world data. Thus, a psychotherapy dataset is analyzed using the model architectures introduced and tested in the previous chapters of the thesis. Firstly, it is shown how Bayesian neural networks can be used to validate the statistical measures used in the field of psychotherapy. Secondly, it is demonstrated how Bayesian neural networks can accommodate a problem of obtaining expert annotated datasets in the field of psychotherapy by leveraging active learning. Thus, chapter 4 initially uses autoregressive Bayesian neural networks to investigate dyadic synchrony in patient and therapist heart rate variability. The Bayesian neural network predictions are shown to be in line with other statistical tests of timeseries similarity and it is proposed that Bayesian neural networks can be used to validate dyadic synchrony in psychotherapy. A subsequent analysis is run to investigate the relationship between dyadic heart rate variability synchrony and patient reported patient-therapist-alliance. Quantifiable measures of dyadic synchrony are used as predictors in an active learning Bayesian neural network that classifies patient reported alliance scores. The model was unable to learn any correlations between dyadic heart rate variability synchrony and alliance from the given data, but it is argued that Bayesian neural networks offer a way to accommodate the costs involved in obtaining high quality psychotherapy datasets. Furthermore, it is proposed that the small sample size in the proof-of-concept study is hugely influential for the model's predictive abilities.

Contents

0 Motivation and Thesis Structure **7**

1 Introduction to Bayesian Neural Networks **9**
1.1 Variational Inference 11
1.2 Reparameterization 12
1.3 Local Reparameterization Trick 13
1.4 Dropout 14
1.5 Bayes by Backprop 15

2 Testing Different Types and Architectures of Bayesian Neural Networks **16**
2.1 Introducing BNN Usecases 16
2.1.1 Alternative Models that are Capable of Supplying Uncertainties 18
2.1.2 Closed Form Bayesian Neural Network Layers Derived from Variational Dropout 19
2.2 Methods 22
2.2.1 The Synthetic Datasets 23
2.2.2 Relaxed No Noise Dataset 27
2.2.3 Experiments 28
2.3 Results 30
2.3.1 Results Best-Case Regression Models 30
2.3.2 Results Best-Case Classification Models 34
2.4 Discussion of the Best-Case Models 41
2.4.1 Choice of Prior 42
2.4.2 The Influence of the Specification of Model Hyperparameters 43

3 Using BNNs for active learning **45**
3.1 Introduction of Active Learning and Oracle Annotaters 45
3.1.1 Oracle Annotators 46
3.1.2 Acquisition Functions 47
3.1.3 Types of Uncertainty 48
3.1.4 Motivation for the Continual Learning Experiments . . 48
3.2 Methods 49
3.2.1 Data 50
3.2.2 Continual Learning Regression Models 51
3.2.3 Continual Learning Classification Models 52
3.3 Results Continual Learning 52

3.3.1 Results Regression Models 52
3.3.2 Results Classification Models 55
3.4 Discussion of Oracle Induced Noise and BNN Resistance to Noise in Continual Learning 61
3.4.1 Contributions and Limitations 62
3.4.2 Efficacy of Actively Queried Datapoints 63
3.4.3 Future Work 64

4 Using BNNs to Validate Statistical Measurements of Dyadic Synchrony 65
4.1 Introducing Psychotherapy and Dyadic Synchrony 65
4.1.1 What is to Gain from Discovering Dyadic Synchrony . 65
4.1.2 Heart Rate Variability Dyadic Synchronization Might Predict Therapeutic Alliance 66
4.1.3 Why Bayesian Neural Networks Provide Unique Validation Power 67
4.1.4 Why Bayesian Neural Networks Can Accommodate Problems in Psychotherapy Research 67
4.1.5 Working Alliance Inventory as a Measure of Therapeutic Outcomes 68
4.2 Methods Psychotherapy Experiments 68
4.2.1 Heart Rate Variability Data 68
4.2.2 Working Alliance Inventory Data 69
4.2.3 Quantification of Timeseries Correlations Using Autoregressive Bayesian Neural Networks 69
4.2.4 Creating Predictor Variables for the Timeseries Regression Model 70
4.2.5 Training a Classification BNN with Active Learning to Predict WAI Scores 70
4.2.6 Dynamic Time Warping 70
4.2.7 Classifying WAI using Measures of Similarity 70
4.3 Results Psychotherapy 71
4.3.1 BNN Models Trained to Find Correlations Between Predictors and Outcome 71
4.3.2 BNNs for Classification of WAI Groups 74
4.4 Discussion of Dyadic Synhrony and BNNs as Statistical Tools in the Field of Psychotherapy 77
4.4.1 Correlation Between Patient and Therapist HRV . . . 77
4.4.2 Classifying Patient Reported WAI Scores Using BNNs 78

5 Conclusion 80

Bibliography **82**

6 Appendix1 Best-Case Models **95**
6.1 Noisy-Data Classification Models Per Class 95
6.2 Relaxed-Data Classification Models Per Class 96
6.3 Single Instance Models 96

7 Appendix2 Continual- and Active Learning Models **103**
7.1 Continual Learning Classification Models Per Class 103
7.2 Active Learning Regression Models 103
7.3 Active Learning Classification Models 105

8 Appendix3 Dyadic Synchrony **107**

0 Motivation and Thesis Structure

In traditional deep learning, models are trained to translate inputs into single point outputs. Given the high-stakes settings these models are regularly used in, it is a big concern that these predictions are made with no uncertainty estimation. Bayesian Neural Networks (BNNs) offer a way to train deep learning models to fit complex and non-linear data while also leveraging the Bayesian framework of integrating prior belief and the likelihood of the given data to produce predictions with uncertainty. This thesis tackles BNNs and contributes research on the mechanics of these models as well as it deploys them on real-world data. The thesis answers the questions: *what are the upsides, shortcomings, and use-cases of BNNs and how could they be used to alleviate data labelling costs involved in expert-domain dataset acquisition?* The thesis tackles these questions in distinct chapters that ultimately conclude by demonstrating tangible use-cases of BNNs in the field of psychotherapy. In chapter 1 the mathematical foundations of how BNNs are implemented are introduced, and this chapter is included in the thesis to guide the reader through the considerations, choices, and complications involved in implementing BNNs. In chapter 2 synthetic datasets are used to train different architectures on large datasets with fully available and annotated datasets for both regression and classification. This chapter demonstrates how BNNs can learn patterns in multidimensional data and is one of the few studies that utilizes BNNs on non-curated benchmark datasets. This chapter also serves the purpose of being a point of reference for the subsequent chapters in the thesis. In chapter 3 the same datasets and models that were tested in chapter 2 are implemented in an active learning pipeline. This chapter demonstrates a tangible use-case of how the uncertainty in BNN predictions can be used for active learning, and the active learning models are compared to the models trained in chapter 2 to highlight the effects of training the models in an active learning pipeline. To the knowledge of the author, it is the only study that evaluates the effects of training BNNs in an active learning environment by comparing the model's performance to equivalent models trained on much larger datasets. This chapter is offering new insights into the effects of using BNNs as the model type of choice in active learning. Chapter 3 is seen as a natural progression of chapter 2, and future work is therefore discussed in chapter 3 only. In chapter 4 the models are used in proof-of-concept experiments to show how BNNs and active learning can be utilized in the field of psychotherapy. This chapter demonstrates how BNNs can be used as validation tools to consolidate findings made by other statistical models, and it shows how BNNs can be used to accommodate problems of acquiring costly data annotations in the field of psychotherapy. The chapters

are distinct and can be read independently, but they progress and build on one-another. Chapter 3 is e.g., intrinsically tied to chapter 4 as it builds its methodology on the findings of chapter 3. All chapters except chapter 1 introduce new terminology and discuss it based on demonstrated experiments. Chapter 1 is a pure introduction and the concepts introduced in this is the building blocks for all the experiments conducted in the thesis at hand.

1 Introduction to Bayesian Neural Networks

Artificial neural networks (ANNs) are widely used for a variety of applications due to their ability to describe high-dimensional and non-linear data (Abiodun et al., 2018). Traditionally, these models incorporate trained parameters in the form of weights and biases. However, ANN's suffer from some limitations in usage and explainability. For instance, overfitting is a common problem to encounter when working with ANN's (Ying, 2019). Furthermore, ANN's do not provide uncertainty estimates for their predictions which makes their predictions unnuanced and potentially harmful (Gawlikowski et al., 2022). Current ANN's are deployed in high-risk areas such as the banking and finance industry, the health sector, pedestrian detection for autonomous vehicles, etc. (Adamu, 2019; Alfred Daniel et al., 2023; Pandit & Garg, 2021). In these applications a wrong decision can be costly, harmful, and detrimental. It is therefore crucial to nuance and quantify how confident ANN's are about certain predictions (Zeng et al., 2022). One proposed way to deal with the lack of uncertainty estimates in predictions is Bayesian Neural Networks (Bykov et al., 2020). These combine central concepts from machine learning and regular neural networks with the Bayesian school of statistics.

Bayesian Neural Networks (BNNs) are analogous to ANN's in most aspects, but they differ in how they treat their network parameters. Where an ANN would consider its weights as single-point estimates, a BNN has a posterior distribution for each of its weights. This offers a way to get uncertainty around the predictions of a network as the network produces predictions for unseen data by sampling from parameter distributions and providing a Bayesian Model Average as the new input passes through the network (Izmailov et al., 2021). Consider a neuron in a regular neural network:

$$f\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{1}$$

Where f is an activation function of the summed inputs x_i multiplied by the weights w_i and the added value of the bias b. In a fully connected feedforward ANN, the weights and the biases for a single layer l can be expressed as matrices. Hence the weights are contained in $N \times M$ matrices where N is the number of neurons in a layer and M is the number of neurons in $l + 1$. Due to the fully connected nature of the network, each output from N is

multiplied by a value in each M. All the weights can therefore be considered numbers in an $N \times M$ matrix. Correspondingly, the biases can be considered numbers in a vector of size M. In a regular neural network, all these numbers are single point estimates that are estimated through gradient optimization techniques (Dogo et al., 2018). Depending on the weight initialization-values preceding training, the weights will be estimates of specific values that are assumed to be approximations of their true values (M. Liu et al., 2023). In a BNN these values are treated as posterior distributions according to Bayes' theorem.

Bayes' theorem demands that the parameters are treated as probability distributions that can be inferred as parameters by incorporating prior information of the distributions with the likelihood of the data and evidence of the data (Bayes, 1991). Thus, a BNN is interested in finding the posterior distribution of the random variable weights θ. Using Bayes' theorem, the posterior probability distribution of θ given the data D can be inferred as:

$$P(\theta|D) = \frac{P(D|\theta)P(\theta)}{P(D)} \tag{2}$$

Where $p(D \mid \theta)$ is the likelihood of generating the data given the parameters.

In the context of BNNs, finding the probability distribution of the evidence $P(D)$ requires inferring the weights by integrating over the likelihood and prior with respect to the weights:

$$P(D) = \int P(D|\theta)P(\theta)\, d\theta \tag{3}$$

However, doing this is mathematically intractable as it would require integrating over all the potentially billions of parameters to find the optimal combination of weight distributions to find the true posterior distribution. Essentially, finding the true posterior distribution over the parameters in a multidimensional and non-linear model, such as a BNN is computationally impossible to do in closed form integration as it relies on finding the probability distribution of the evidence, which in turn is dependent on the likelihood and prior distributions:

$$p(\theta|D) = \frac{p(D|\theta)p(\theta)}{\int p(D|\theta)p(\theta)\,d\theta} \tag{4}$$

Different approximation techniques have been applied to solve the issue of analytical intractability of the integration with respect to the model parameters in the Bayesian evidence term. Most commonly, simulation in the form of Markov Chain Monte Carlo or optimization with variational inference have been applied (Izmailov et al., 2021; Mostafa et al., 2023)

This thesis will consider BNNs using variational inference due to practical reasons and available computational power. However, the reader should note that some authors, e.g., Izmailov et al. (2021) have argued for the use of Hamiltonian Monte Carlo sampling whenever computationally possible, and Blei et al. (2017) stress that using MCMC guarantees asymptotic results while the computational benefits of using VI is likely to only yield results relatively close to the true posterior.

1.1 Variational Inference

Variational Inference (VI) is a method widely used to make BNNs computationally tractable (Hønsi, 2023; Izmailov et al., 2021; Zhan et al., 2020). The notion behind VI is to use a collection of tractable distributions in a controlled, optimizable distribution space to infer the true posterior distribution. Thus, using variational inference to find a distribution q_ϕ where $q_\phi \in Q$ in which Q is a collection of variational distributions that are optimized to infer the true posterior P, can reduce the problem of finding the posterior distribution to an optimization problem. Effectively, this means finding the optimal parameters ϕ to minimize the distance between the true posterior distribution $p(\theta|D)$ and the variational distribution q_ϕ.

Finding the best variational distribution is done by minimizing the Kullback-Leibler Divergence (henceforth "KL Divergence") between $p(\theta|D)$ and q_ϕ. This is expressed mathematically:

$$\mathrm{KL}\left[q(\theta)\|p(\theta|D)\right] = \int q(\theta)\log\frac{q(\theta)}{p(\theta|D)}\,d\theta = E_{q(\theta)}\left[\log\frac{q(\theta)}{p(\theta|D)}\right] \tag{5}$$

Finding the true posterior is a question of finding the variational distribution

that minimizes the divergence between the true posterior the best variational distribution:

$$q_\phi^*(\theta) \in Q \tag{6}$$

Mathematically expressed:

$$q_\phi^*(\theta) = \underset{q_\phi^*(\theta) \in Q}{\arg\min} \, \mathrm{KL}\left[q(\theta) \| p(\theta|D)\right] \tag{7}$$

Implementing the KL-Divergence is at this point still intractable because it relies on minimizing the distance between a variational distribution and the true posterior. The true posterior is unknown, and therefore the equation must be rewritten further. As discussed in Blei et al. (2017), Hønsi (2023), and Rakesh and Jain (2021) the problem can be reduced to an optimization function where the objective is to maximize the evidence lower bound (ELBO):

$$L(\phi) = E_{q_\phi(w)}\left[\log p(D|w)\right] + \mathrm{KL}\left[q_\phi(w) \| p(w)\right] \tag{8}$$

Where the first term is the data-dependent likelihood term and the second term measures the KL divergence between the prior and posterior.

1.2 Reparameterization

Because the posterior of the weights in a variationally inferred BNN are approximated using a parametric distribution seeking to minimize the KL divergence, and the optimal value of the parameters used in the variational distributions in the KL Divergence are estimated by maximizing the evidence-lower bound of the variational distributions, approximating the posteriors in a VI BNN can be reduced to a computationally tractable task (Molchanov et al., 2017). However, even when reduced to maximizing the ELBO, estimating the expectations of the variational distributions in equation 8 can prove to be a computationally intractable task because performing stochastic gradient descent depends heavily on the variance of the gradients (Kingma et al., 2015; Molchanov et al., 2017). Therefore, estimating the gradients of the variational distribution's gradients is done by using the "reparameterization

trick", in which mini-batches are deployed to compute stochastic estimates of the ELBO, and thereby achieving a reduction in the variance of the ELBO gradients (Kingma et al., 2015; Molchanov et al., 2017). Kingma and Welling (2013) show how the reparameterization trick can be used to get an unbiased differentiable minibatch-based Monte Carlo estimator of the expected log-likelihood seen in equation 8. The key idea is to cast parametric noise $q_\phi(w)$ over the variational distributions as a deterministic differentiable function $w = f(\phi, \epsilon)$, where ϵ is a random variable sampled from a distribution $\epsilon \sim p(\epsilon)$, see Kingma and Welling (2013) for a full mathematical walkthrough of the reparameterization trick. All in all, reparameterization is done to make stochastic gradient descent feasible in a variational inference Bayesian Neural Network.

1.3 Local Reparameterization Trick

A method to further reduce the variance of the gradient estimator, is to use the "local reparameterization trick". Here the intuition is to sample different weight matrices for each datapoint inside the minibatches and make the computation feasible by moving the noise from the weights to the resulting activations (Molchanov et al., 2017). By assuming independence between the variances in the minibatches, or in Kingma et al.'s Kingma et al. (2015) terminology; by having the covariance of the expected log likelihood in each minibatch be 0, the variance of the stochastic gradients in the Bayesian optimization process scales to $1/M$, where M is the number of minibatches used. This means that the uncertainty in all the global noise for all the weights in a network can be reinterpreted locally, so that instead of drawing separate weight matrices for all the training examples in M, the activations can be sampled directly.

Consider a hidden layer with k units receiving an input feature matrix A, where $A = k \times M$. Before applying the local reparameterization trick, the weight matrix of size $k \times k$ would have to be multiplied before any activation functions would be applied to the inputs. To keep the covariance of all examples in the minibatch independent, this would entail sampling $M \times k \times k$ random numbers. Moreover, passing the resulting outputs as inputs to a new hidden layer after having applied a non-linear activation, would massively multiply the number of computations needed (see Kingma et al. (2015) section 2.3).

For a fully factorized Gaussian, $q_\phi(w_{i,j}) = \mathcal{N}(\mu_{i,j}, \sigma^2_{i,j}) \, \forall w_{i,j} \in W$, the weights are sampled as $w_{i,j} = \mu_{i,j} + \sigma_{i,j}\epsilon_{i,j}$, with $\epsilon_{i,j} \sim \mathcal{N}(0, 1)$. Because the posteriors

of the weights are a factorized Gaussian, the posteriors of the resulting activations are also a factorized Gaussian. The local reparameterization trick utilizes this to sample the posteriors of the activations from their implied Gaussian distributions directly, and thus saves the computational cost involved in sampling the weights and then applying activations. In addition to reducing sampling to the activation stage of the model inputs' pass through the network, using the local reparameterization trick also reduces the variances of the gradient estimator (Kingma et al., 2015).

1.4 Dropout

A traditional neural network can reduce its parameter space by utilizing dropout in which random nodes in the network are set to zero or "dropped out" in different realizations of the model (Srivastava et al., 2014). Originally, dropout was introduced in the form of binary dropout in which a Bernoulli distribution was used to sample a probability, called the dropout rate, for whether units in a hidden layer should be set to zero (Hinton et al., 2012). Later a Gaussian interpretation was introduced in which continuous noise is multiplied on inputs being passed through a network (Srivastava et al., 2014). This is equivalent to putting Gaussian noise on the weights and this method can be used to gain a posterior distribution over the weights (Kingma et al., 2015; Molchanov et al., 2017; Wang & Manning, 2013). Kingma et al. (2015) show how dropout is a special case of Stochastic Gradient-based Variational Bayes where the local reparameterization trick is used to find uncertainty locally for each datapoint in a minibatch, and they use this to propose inferring dropout rates directly from the data rather having them be fixed prior to training. Inferring directly from the data is a specific method of what the authors call variational dropout, in which the specific data controls the dropout rates adaptively. In variational dropout, the dropout rates become parameters to be learned in the optimization of the variational distributions involved in Stochastic Gradient-based Variational Bayes. Inferring the rates in variational dropout involves setting a log-uniform prior on the model's weights, and this has been critiqued for being theoretically ill-posed (Y. Liu et al., 2019). The main critique point of the log-uniform prior is that it is improper with its integral being infinite which can result in the variational inference of model weights being degenerated into penalized maximum likelihood estimation. Y. Liu et al. (2019) offer an extension to variational dropout that they call variational Bayesian dropout in which they use a hierarchical prior for the network weights. All in all, variational dropout and its extensions offer a way to incorporate dropout into a Bayesian deep learning setting.

1.5 Bayes by Backprop

Bayes by Backprop was created as a way to learn probability distributions over the weights in neural networks while utilizing stochastic gradient descent (Blundell et al., 2015). Its implementation involves using a noise random variable ϵ in a deterministic function $t(\theta, \epsilon)$, and using the function to transform a sample of the noise ϵ and the variational posterior parameters θ into a sample from the variational posterior. Blundell et al. (2015) achieve this by proposing:

"Let ϵ be a random variable having a probability distribution given by $q(\epsilon)$ and let $w = t(\theta, \epsilon)$ where $t(\theta, \epsilon)$ is a deterministic function. Suppose further that the marginal probability density of w is $q(w|\theta)\, dw$."

And by using this proposition in the optimization problem of maximizing the ELBO (see equation 8), Blundell et al. (2015) can evaluate the expectations by using Markov chain Monte Carlo to draw samples from the variational posterior. The resulting algorithm that optimizes, thus does so iteratively, and tries to sample the expectations that minimizes the KL divergence between the variational posterior distributions and the true distributions, and this algorithm thereby achieves a very backpropagation like approach to learning the distributions over the weights in a BNN. Note that the proposition given by Blundell et al. (2015) is the reparameterization trick already introduced in this thesis (Kingma et al., 2015; Kingma & Welling, 2013). The novel contribution of Blundell et al. (2015) is that they apply their Bayes by Backprop algorithm to weights instead of hidden units. Furthermore, they do not use a closed form of the complexity cost (the prior dependent term in equation 8), and thereby allow for a broader palette of combinations of prior and variational families. The Bayes by Backprop algorithm has been used in many practical implementations such as: transport modes recognition Jia et al. (2020), management of dialogue for statistical spoken spoken dialogue systems Tegho (2017), and a word embedding model (Han et al., 2018).

2 Testing Different Types and Architectures of Bayesian Neural Networks

The purpose of this chapter is to demonstrate the usability and predictive performance of different Bayesian Neural Networks. In this chapter models are trained and tested on a synthesized dataset. The advantage of using a generated dataset is to know exactly which patterns are underlying the data structure. This is particularly useful when testing different models, model architectures, and model specifications because it is naturally highlighted if reducible noise is introduced by controllable and changeable model specifications (Jordon et al., 2020). Furthermore, because the dataset is synthetically engineered, it is much safer to assume causality between the patterns found by the models tested in the thesis. This is because knowing the generating mechanism behind the data allows for evaluating whether the models find the correlations that are compatible with the causing generating mechanism. Moreover, even if patterns should exist in the data that are outside the realms of the synthetic generating mechanism, they would be spurious, or not causal, and therefore not important to identify with the models.

Considering that the end goal of this thesis is to demonstrate the usability of BNNs in a complicated cognitive dataset, the synthetic dataset is created to simulate a cognitive task. This is helpful in multiple ways for reaching the ultimate ambition of using BNNs in an active learning pipeline to understand a psychotherapy dataset. Firstly, simulating a dataset that closely resembles possible human behavior allows for monitoring model behavior in a closed, experimental environment. Secondly, creating a behavioral dataset allows the reader to follow the logic and applicative power of applying Bayesian deep learning models more easily to a dataset containing human behavior variables. Lastly, to make chapter 3 of this thesis more tangible, it is important that the data is easy to annotate by either a human expert or an otherwise qualified oracle (see chapter 3 about continual learning and oracle models). Synthesizing the data to show human behavior-like patterns, makes this effort more straightforward. In conclusion, it is decided to utilize a synthesized human behavior dataset to demonstrate the usability and performance of different Bayesian Neural Networks.

2.1 Introducing BNN Usecases

Traditional neural networks are ubiquitous in machine learning implementations and have been used for a variety of tasks. However, some of the biggest shortcomings of these models is that they do not provide uncertainty

estimates for their model predictions and that they are prone to overfitting. Bayesian Neural Networks, like regular neural networks, have been successfully implemented in several traditional machine learning tasks. Tasks include regression and classification, but are not limited to these, and have been used in e.g., time-series modelling, out-of-distribution detection and active- and continual learning (Gawlikowski et al., 2023). As opposed to regular neural networks, they are widely recognized for their ability to supply uncertainty estimates around both their model parameters and their predictions (Bykov et al., 2020). Furthermore, their Bayesian nature makes them more robust against overfitting problems (Fortuin et al., 2022). BNNs are probabilistic models that in their traditional implementation utilize labelled data to perform posterior inference (Sharma et al., 2023). BNNs have been used in practical settings for a variety of tasks. They have e.g., been used for flood forecasting (Zhan et al., 2020). In this contribution the authors identify that floods have disastrous consequences for people who live in affected areas. They, furthermore, scrutinize the impacts of flood forecasts and their potential massive humanitarian benefit, but they also state that errors in these forecasts can be fatal. They introduce BNNs as a way of quantifying the uncertainty in the forecasts. By demonstrating the mathematical implementations of variational inference models and by evaluating implementations on a variety of evaluation metrics, the authors conclude that BNNs outperform competing models such as Hidden Markov Models and Gaussian Process Regression both in terms of the reliability of model outputs and in the quantification of prediction uncertainties.

BNN's have also been implemented in a psychological setting where their learning capabilities have been compared to that of humans' (Binz & Endres, 2019). By operationalizing different levels of human experience with different sizes of training datasets, i.e., larger datasets represent older infants than smaller datasets, it is shown that humans share learning characteristics with deep learning models. This is a more philosophically grounded implementation of BNNs because the implementation rests on psychological theories that state that human learning stages are initially guided by simple rules before more complex systems take over as learning progresses. The justification for using BNNs is therefore based on their ability to approximate an arbitrarily complex mapping given enough capacity while simultaneously incorporating initial belief updating vis-à-vis new evidence. Even though the authors hypothesize that other function approximators in combination with Bayesian learning could have led to similar results, their implementation shows how BNNs can be used for their inherent prior updating qualities.

2.1.1 Alternative Models that are Capable of Supplying Uncertainties

BNNs are a great tool for getting quantifiable uncertainty in model predictions. This has been shown to be beneficial in several machine learning tasks including biomedical imaging, hydrological forecasting, and molecular property prediction (Kwon et al., 2020; Ryu et al., 2019; X. Zhang et al., 2009). However, other model types can supply quantifiable uncertainty for their predictions. Ensemble models have been shown to be decent providers of uncertainty quantification and enhance the model's predictive performance (Abdar et al., 2021). In ensemble modelling multiple models are trained and their predictions are combined. This approach is generally used across machine learning tasks and is considered state of the art for many tasks (Sagi & Rokach, 2018). Typically, predictive uncertainty is achieved in ensemble settings by training the same architecture with different weight instantiations and then using some form of acquisition technique to quantify the uncertainty estimates (Lakshminarayanan et al., 2017). Another widely used way of achieving uncertainty is by using dropout neural networks (see chapter 1 for a more in detail explanation of dropout). A dropout neural network sets some of its, but different ones for different realizations, weights to zero and thereby achieves different outputs for different realizations of the network. By realizing the network multiple times, the same inputs will manifest as different outputs and thereby provide a group of predictions to quantify uncertainty estimates from (Gal & Ghahramani, 2016). Notably, Gal and Ghahramani (2016) showed that a typical optimization of regular neural networks with dropout layers is equivalent to BNNs using variational inference with a specific variational distribution. They show that the dropout objective of a network minimizes the KL divergence between an approximate distribution and the posterior of a deep Gaussian process. In J. Liu et al. (2019) the authors test Bayesian non-parametric ensembles to illuminate their models' predictive performance and the uncertainties related to those. Furthermore, they do so to show where uncertainty in the model predictions arises due to aleatoric and epistemic uncertainty (see chapter 3 for an introduction of different types of uncertainty). They find that their models' uncertainty can be accurately traced to their source under complex observational noise. All in all, there are different readily available methods for achieving uncertainty estimates for model predictions.

Even though methods such as ensemble models, dropout neural networks, and various Bayesian approaches achieve quantifiable uncertainty estimates for their predictions, BNNs have been argued to be the most robust, available method for achieving this end. BNNs have e.g., been shown to capture

epistemic uncertainty more efficiently than regular neural network ensembles (Dusenberry et al., 2020). Furthermore, they have been shown to have robust uncertainty estimation on well-known image classification datasets (Mitros & Mac Namee, 2019). When using mean-field variational BNNs, i.e., a fully factorized Gaussian variational distribution $q_\phi(\theta)$, it has been shown that BNNs fall short of correctly classifying *"in-between"* datapoints i.e., datapoints that are found in between separated regions of observations (Foong et al., 2019). This can lead to catastrophically overconfident predictions on out-of-distribution data. A linearized Laplace approximation is proposed for small network architectures as an alternative. In sum, multiple methods achieve quantification of uncertainty for model predictions, but BNNs arguably provide the theoretically most robust and elegant approach to achieving this goal. BNNs have shortcomings such as having overconfident predictions on out-of-distribution data and being computationally expensive to train.

2.1.2 Closed Form Bayesian Neural Network Layers Derived from Variational Dropout

As presented in chapter 1, BNNs have made use of various forms of regularization techniques, such as reparameterization, local reparameterization of global noise, and dropout. In a paper on BNNs and computational compression, the reparameterization of Molchanov et al. (2017) that utilizes variational dropout with three fixed dropout rates α, is combined with the local reparameterization trick presented by Kingma et al. (2015) to achieve superior BNN compression without trading off model performance (Federici et al., 2017). This closed form neural network utilizes variational dropout to make an easily implementable framework for Bayesian deep learning. Moreover, this network uses additive noise in its reparameterization as well as it incorporates the local reparameterization trick. Using this implementation, a BNN framework for active learning has been demonstrated (Rakesh & Jain, 2021). The framework proposed by Rakesh and Jain (2021) is implemented to exhibit BNN model performance (chapter 2 of this thesis), and to showcase the usability of BNNs in continual learning (chapter 3 of this thesis), as well as to demonstrate real-world applicability of BNNs in active learning on a psychotherapy dataset (chapter 4). This framework is chosen due to its computational efficiency and theoretical justification made clear by numerous advances within the field of Bayesian Deep Learning (see chapter 1 of this thesis). Different model architectures are tested with the closed form variational dropout layers in the current chapter.

The closed form variational inference layers are implemented as such: Recall equation 8 in which the second term is the regularizer that minimizes the KL-divergence between the prior and the posterior:

$$KL\left[q_\phi(w) \parallel p(w)\right]$$

(8a) (second term of equation 8)

Like in Rakesh and Jain (2021), the closed form variational inference models in this thesis utilize the scaled normal prior proposed in (Louizos et al., 2017). This choice is motivated by this prior being a continuous relaxation of the spike-and-slab distribution (Mitchell & Beauchamp, 1988). The prior is set over parameters in the BNN w:

$$\begin{aligned} &w \sim \mathcal{N}(w; 0, z^2), \text{ with its scales being treated as random variables:} \\ &z \sim p(z) \end{aligned} \tag{9}$$

As explained in Louizos et al. (2017) the fact that the scales are treated as random variables makes it possible to recover marginal distributions over the parameters with heavier tails and denser mass at zero. The mixing density of the scales in the spike-and-slab distribution is a Bernoulli distribution, so that the marginal $p(w)$ has a delta spike at zero and a continuous slab over the real line and can be interpreted as a special case of Monte Carlo Dropout. This is a logical consequence of setting a non-arbitrary amount of the weights to zero as a spike at zero in the marginal $p(w)$ will naturally occur under these conditions. The scales of the proposed normal prior follow a log-uniform prior $p(z) \propto |z|^{-1}$, and this allows for recovery of the log-uniform prior over the weights if marginalizing over the scales:

$$p(w) \propto \int \frac{1}{|z|} \mathcal{N}(w \mid 0, z^2)\, dz = \frac{1}{|w|} \tag{10}$$

This parametrization is proposed by Figueiredo, (2001) and is known as a normal-Jeffrey's prior. This parameterization makes sharing the scales across

connected units in the network feasible, i.e., $p(w)$ for a neuron and its belonging feature map because the scales can be shared in their joint prior distribution. Consider the probability of the shared scales in a weight matrix W for a fully connected neural network with input dimensions n and output dimensions m:

$$p(W, z) \propto \prod_{i}^{n} \frac{1}{|z_i|} \prod_{ij}^{m,n} \mathcal{N}(w \mid 0, z^2) \tag{11}$$

It becomes feasible to use variational inference with a joint approximate posterior:

$$q_\phi(W, z) = \prod_{i=1}^{n} \mathcal{N}(z_i \mid \mu_{z_i}, \mu_{z_i}^2 \alpha_i) \prod_{i,j}^{m,n} \mathcal{N}(w_{ij} \mid z_i \mu_{ij}, z_i^2 \sigma_{ij}^2) \tag{12}$$

In this equation α_i is the dropout rate for the group in question and the multiplicative parameterization of the approximate posterior can be re-parametrized further (the first term in equation 12), such that $\sigma_{z_i}^2 = \mu_{z_i}^2 \alpha_i$ and the following ELBO looks like this:

$$L(\phi) = E_{q_\phi(z) q_\phi(W|z)} \left[\log p(D \mid W)\right] - E_{q_\phi(z)} \left[\mathrm{KL}(q_\phi(W \mid z) \parallel p(W \mid z))\right] - \mathrm{KL}\left(q_\phi(z) \parallel p(z)\right) \tag{13}$$

The KL divergences from equation 13 can be replaced with closed form expressions:

$$\begin{aligned}
&\mathrm{KL}(q_\phi(W \mid z) \parallel p(W \mid z)) = \frac{1}{2} \sum_{i,j}^{m,n} \log \left(\frac{e^{\sigma_{ij}^2 + \mu_{ij}^2 - 1}}{\sigma_{ij}^2} \right), \\
&\text{and} \\
&\mathrm{KL}(q_\phi(z) \parallel p(z)) \approx \sum_{j}^{n} k_1 \left(1 - \gamma(k_2 - k_3 \alpha_j) - \frac{m(\alpha_j)}{2k_1} \right),
\end{aligned} \tag{14}$$

in which $\alpha_j = -\log \left(\frac{\sigma_{z_j}^2}{\mu_{z_j}^2} \right)$, $\gamma(\cdot)$ is the sigmoid function, and $m(\cdot)$ is the softplus function (note that m denotes differently here than in equation 13. Notation is chosen to best comply with typical notation standards). The constant dropout rates approximated by Louizos et al. (2017) are: $k_1 = 0.63576, k_2 = 1.8732, k_3 = 1.48695$. Using the closed form KL-divergences means that the ELBO can be optimized by using traditional stochastic gradient descent.

2.2 Methods

Three different models were tested on a regression task and three models of similar architectures were adapted to fit a classification task. The models were tested to achieve baseline knowledge of learning behavior for different model architectures, before they were tried in an active- and continual learning pipeline in chapter 3 of this thesis. The models were all tested on the same synthetic datasets with continuous outcomes being proposed in a noisy and relaxed version. Datasets with continuous outcomes are henceforth referred to as "regression datasets". Conversely, "classification datasets" were adapted versions of the regression datasets made suitable for classification tasks, i.e., with categorical outcomes. The models were tested on both a regression and classification task to show the benefits of BNNs in both cases. Due to computational restraints, all the model architectures were small, fully-connected neural networks. Two model architectures were implemented using the closed-form layers discussed in the introduction section of this thesis, and one model was implemented using Bayes by Backprop. This was the case for both the regression models and the adapted classification models. In the current chapter of this thesis, the best-case models are presented. These are models that were trained using all the available synthesized data. Because the data is synthesized it was possible to train the model on all the data with all the data being annotated prior to training. These models can be thought of as best-case models because the training procedure emulates a scenario in

which a dataset is obtained and fully annotated by an expert. These models were theoretically the most desirable models to train, because they emulated a scenario in which active or continual learning is not necessitated because a large, annotated dataset is readily available. The models were trained and shown here as a guideline for comparison in the discussion in chapter 3 of this thesis, in which a scenario with far fewer annotated datapoints is emulated. In sum, three architectures were tested on a noisy and relaxed dataset in both regression- and classification implementations, making the total number of implemented best-case models 12.

2.2.1 The Synthetic Datasets

Human behavior is complex and in many cases unpredictable, however general patterns exist and deep learning models have been used to gain insights into human behavior (Fintz et al., 2022). The datasets used for analyses in the current chapter of this thesis were created with inspiration from human decision-making behavior. Thus, the data was created from the hypothesis that human grocery behavior is variable and influenced by different quantitative features. This seems the likely case when considering results from Pechey and Monsivais (2015) in which socioeconomic status is shown to influence the quality of food purchased by different individuals. In fact, many different socioeconomic factors influence grocery shopping behavior (Darko et al., 2013; Pechey & Monsivais, 2015; Ricciuto et al., 2006; Turrell & Kavanagh, 2006). The dataset was synthesized to contain a variable called *"quality of food purchased"* as a numeric measure between 1 and 5. Predictor variables included were: *"monthly income"*, *"time of month"*, *"savings"*, and *"guests"*. *"Monthly income"* was generated from a discrete uniform distribution and specific values range from 1000 to 10000 and reflected a monthly income in a non-specified currency. *"Time of month"* was introduced to capture human consumer behavior of spending more on goods earlier in the month closer to when salary is paid out. *"Time of month"* was sampled from a discrete uniform distribution as integers between 1 and 31 with integers representing days of the month. *"Savings"* was also generated from a uniform distribution and was generated to be one of "low", "medium", or "high". *"Savings"* was created to make sure that total net worth was considered as a predictor for the quality of food purchased in cases where salary was low, but the individual financial situation might have made buying high quality food feasible. *"Guests"* was a variable that considered whether the person purchasing food was having guests and thereby purchasing food for not only themselves, but for other people as well. This variable was generated under the presumption that having guests would entail purchasing food of better

quality. Guests was generated from a binomial distribution and had probability 0.1 for guests being present. Furthermore, the outcome variables were sampled with an artificially introduced noise parameter. The variables were used to synthesize data with using the logic in algorithm 1.

Algorithm 1: Algorithm to synthesize Grocery Data

Inputs : income, time, savings, guests
Output: quality_of_food
Let QBI be quality of food based on income, QBT be quality of food based on time of month, QBS be quality of food based on savings, and QBG be quality of food based on guests being present
QBI $\leftarrow$ 5 - `floor` (`min` (income, 8330) / 1666)
QBT $\leftarrow$ `if` (time $\geq$ 16, 1, 5)
QBS $\leftarrow$ `if` (savings == "high", 5, `if` (savings == "medium", 3, 1))
QBG $\leftarrow$ `if` (guests == False, 3, 5)
quality_of_food $\leftarrow$ (QBI * 0.4 + QBT * 0.1 + QBS * 0.2 + QBG * 0.3)
noise $\sim$ `normal` (0, 1)
quality_of_food += noise
quality_of_food = `clip` (quality_of_food, 1, 5)
`if(`$classification_dataset$`)`$quality_of_food$ = `minority_oversampling`($quality_of_food$)
return *quality_of_food*

Algorithm 1: the general logic used to synthesize data with

The data therefore considered different factors that could potentially drive consumer behavior. Two different versions of the dataset were made. One where the quality of the food was floating point numbers, i.e., decimal numbers, and one where the quality of the food was represented by integers. These were made for regression and classification tasks respectively. See figure 1 for a visualization of the simulated data. Because the data generating mechanism sampled its Y-values from a defined equation in which certain values of X made it more likely to end up with higher class values for Y, the classes might not entirely reflect distinguishable categories. E.g., specific X-values might warrant that the quality of food is high, but whether the quality is 4 or 5 could be arbitrary. A case could therefore be made that it would make more theoretical sense to dichotomize the categories and

make the classes be e.g., "low", and "high". However, the classification data was kept as it was, for two reasons. Firstly, whether the models achieved good evaluation metrics was not pivotal for demonstrating the applications of BNNs. This is because anything above chance level would suffice to argue that BNNs can supply uncertainty around predictions. What was ultimately important to demonstrate was the reliability of the uncertainty estimates and their functionality in a continual learning context. Secondly, having the dataset be challenging to classify while testing the models in the closed experimental setting made it more probable that any misclassifications due to misspecification of the models, could be caught before testing the models in a real experimental environment, i.e., the models needed to be specified correctly to achieve any useable evaluation metric in the closed environment, when the data was hard to classify. All in all, the datasets considered the predictors *"monthly income"*, *"time of month"*, *"savings"*, and *"guests"* for the dependent variable *"quality of food purchased"* in both regression and classification datasets and in noisy and relaxed versions.

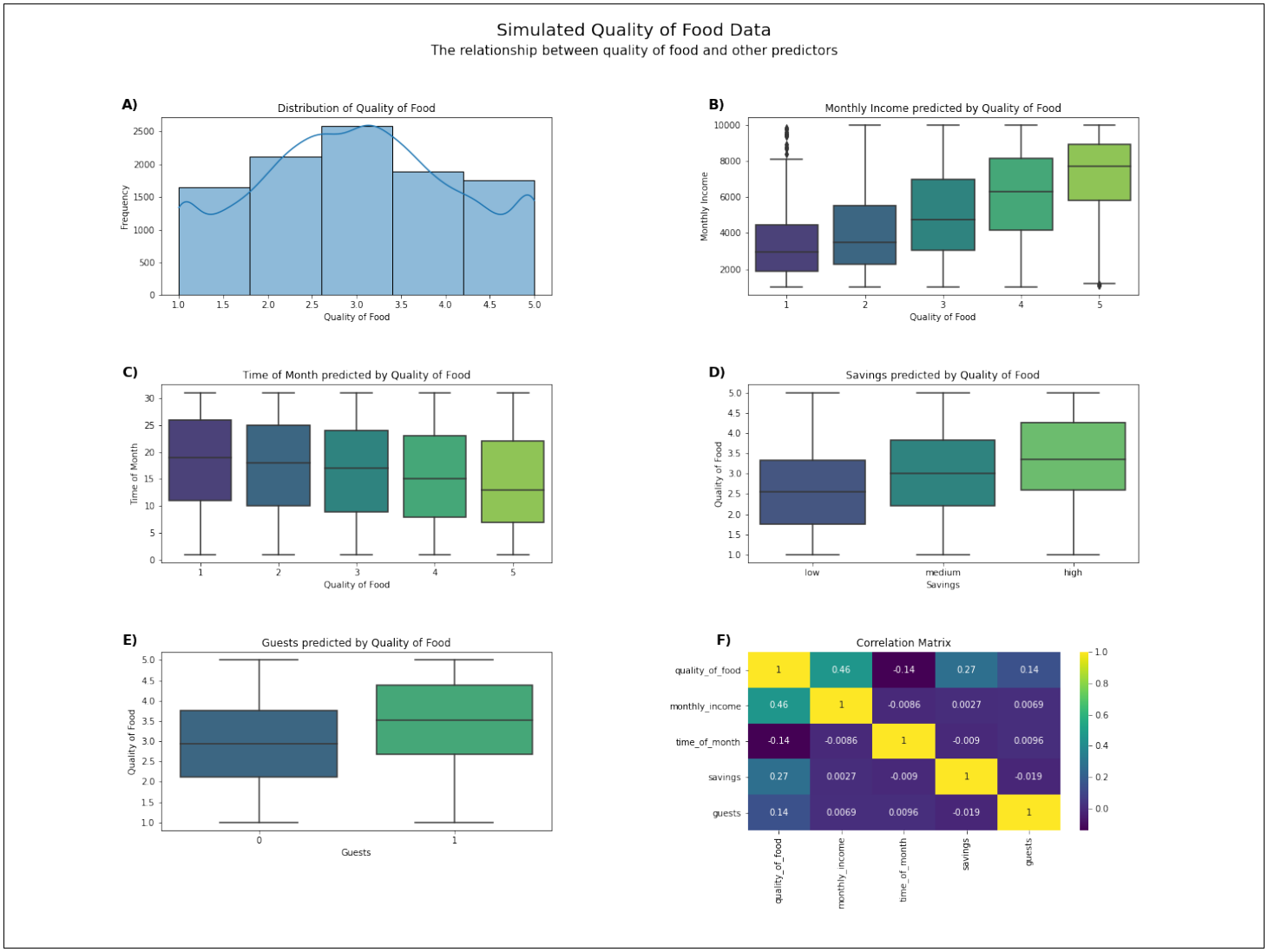


Figure 1: shows the simulated grocery shopping data. Note that the data that is visualized is the regression data, i.e., the data where quality of food is floating point numbers. However, for visualization purposes some of the subpanels have binned quality of food into categorical levels. These levels have been binned the same way the data was made fit for classification, i.e., by a simple rounding to nearest natural number. A) shows the distribution of the quality of food purchased for the whole dataset. Each histogram bin shows 20% of the total observations starting from 1 going to 5. The density line shows the general distribution of the observations not contained in bins. B) shows boxplots of monthly income for each level in the categorical version of the data. C) shows boxplots of time of month for each level in the categorical version of the data.

(continued) D) shows boxplots of quality of food for each of the categorical levels in savings (note that quality of food is now on the y-axis). E) shows boxplots of quality of food for the binary levels in guests with 0 being no guests and 1 being guests present (note that quality of food is now on the y-axis). F) shows a correlation matrix for all the variables, lighter colors illustrate a high positive relationship between variables while darker colors represent minimal or negative correlation. (continued on next page)

It should be clear to the reader from figure 1 that the simulated grocery shopping data had some apparent patterns that reflect the simulation conducted with algorithm 1. Thus, there were clear patterns of *"monthly income"* and *"savings"* having a positive influence on *"quality of food"*. Similarly, a negative effect was observed for *"time of month"* with dates later in the month influencing *"quality of food"* to be worse than earlier in the month. Furthermore, having guests made the quality of food go up. Lastly, it is worth noting that the predictors influenced the quality of food with differing potency, *"monthly income"* was e.g., a bigger influencer than *"guests*, see figure 1 panel F for a detailed illustration of the strength of the individual predictors. The different potencies of the predictors were in line with the equation in algorithm 1 used to simulate quality of food with. The simulated data was therefore a desired mix of predictor variables and noise that reflected the behavior of a real cognitive task.

2.2.2 Relaxed No Noise Dataset

It has been known for many years that shallow neural network architectures with few hidden units and few hidden layers are easy to train for generalization, but that their simplicity comes with a tradeoff of high prediction errors on non-training data (Hunter et al., 2012). All the models trained in this thesis are simple feedforward neural networks with just two hidden layers and no more than 300 hidden units for the biggest layers. A challenge in the model training process is therefore to get the models to learn the patterns in the training data. Considering the complexity of the data generating process, it might be difficult to obtain good predictive results, and a relaxed version of the data was therefore created to test model performances on differing levels of complexity of training data. The relaxed data was synthesized using the same approach as in algorithm 1, with some significant changes. Firstly, *"quality of food"* was sampled directly as a consequence of *"monthly income"*, in essence making any correlations between the other predictors and *"quality of food"* spurious. Secondly, the data was created with no noise. Lastly, the predictor *"guests"* was sampled with a Bernoulli distribution with a probability of 0.3 instead of 0.1 for all observations of *"monthly income"*

in the 70th and above percentile. *"guests"* was sampled this way to create a known spurious relationship between one of the predictors and *"quality of food"*, i.e., *"quality of food"* had a positive relationship with *"guests"* when *"guests"* were present, but only because it was more likely that guests were present for bigger values of *"monthly income"*. This spurious relationship was introduced to not make the datasets completely vanilla even in the relaxed versions. For the regression version of the relaxed dataset, the target values were initially sampled as integer values between 1-5. However, to make the dataset fit for regression the target variables were spread out by sampling them from a normal distribution with a low standard deviation (sd = 0.5). Thus, creating target variables that were floating point numbers while simultaneously inducing a little stochastic noise in the relaxed regression dataset. The models are tested on this dataset as a control for making sure that the models are capable of minimizing aleatoric uncertainty, when epistemic uncertainty in the training data is extremely low.

2.2.3 Experiments

Experiments were performed on different architectures for both regression and classification models. Architectures included both Bayes by Backprop and closed form layers. Experiments were conducted to get clear insights into the performance of BNNs for both regression and classification tasks on datasets of differing complexity. Experiments were conducted on the noisy grocery shopping data and on the relaxed grocery shopping data. For regression tasks R2 and RMSE scores are reported. For classification tasks an accuracy score and an F1-score are reported. Bayes by Backprop (BBB) models were built in Python 3.11.4 using the BLiTZ module that allows for building Bayesian Neural Network layers on top of PyTorch models (Esposito, 2024; Paszke et al., 2019). These models use the scale mixture prior proposed in (Blundell et al., 2015). This prior is fixed form and does not update its hyperparameters during training. Blundell et al. (2015) argue that their choice of this prior was because of its resemblance to the spike-and-slab prior and because their implementation allowed for the prior's parameters to be shared for all the weights in the network. This, they argue, makes it easier to use in the stochastic gradient descent that is utilized in BBB. The scale mixture prior used in the BBB models took this form:

$$P(w) = \prod_j \pi \mathcal{N}(w_j \mid 0, \sigma_1^2) + (1 - \pi)\mathcal{N}(w_j \mid 0, \sigma_2^2) \tag{15}$$

In which w_j is the jth parameter in the network, and $\mathcal{N}(x \mid \mu, \sigma^2)$ is the Gaussian density evaluated at x with mean μ and variance σ^2. σ_1^2 and σ_2^2 are the variances of the mixture components. For all the BBB models implemented in this paper, $\pi = 0.5$, $\sigma_1^2 = 1$, and $\sigma_2^2 = 0.0025$.

Closed form variationally inferred dropout models were implemented from scratch, with the code provided by Rakesh and Jain (2021) in their public GitHub repository. All the models implemented variational inference by finding the Kullback-Leibler divergence between the variational posteriors and the true posterior, and by reducing the parameter estimation process to minimizing the evidence lower bound (ELBO).

Regression Models
Three different model architectures were trained with the regression datasets. Firstly, a simple feedforward neural network was implemented. This model consisted of an input layer, two hidden layers, and an output layer. Hidden layers counted 10 and 20 nodes respectively. This model is hereafter referred to as the "simple" model. Secondly, a bigger feedforward model was implemented with the same number of layers. However, hidden layers contained 300 and 100 units in this model, and this model was an implementation of the well documented LeNet-300-100 architecture Lecun et al. (1998). This model is referred to as the "LeNet-300-100" model for the remainder of the thesis. The two first models used the closed form neural network layers that is derived from variational dropout described in the introduction of the current chapter. Lastly, a model was implemented using the standard supplied Bayesian linear layer from the BLiTZ module which leverages Bayes by Backprop. The last model had the same number of hidden layers and nodes as the second model but shall henceforth be referred to as "BBB". All models' hidden layers used ReLU activation functions.

Classification Models
The three same model architectures trained on the regression datasets were trained on the classification datasets. The model architectures were identical to the regression architectures except for the output layer utilizing a softmax activation function to get outputs as probabilities.

2.3 Results

2.3.1 Results Best-Case Regression Models

Regression models were run on the regression versions of the grocery shopping data. The three model architectures; BBB, simple, and LeNet-300-100 were trained on both the noisy data and the relaxed data. The models were trained as "best-case" scenario models, i.e., they were trained with the whole annotated dataset available. The data comprised 10000 labelled observations for both the noisy and the relaxed data. For the best-case models 30 instances of the network were used to compute uncertainty in the model parameters. However, only a single prediction was made when testing the network, i.e. only a single sample from the posterior distributions of the weights was used to get a prediction with. In chapter 3 a data selection function is implemented to get uncertainties in the model predictions. For the BBB model 3 samples are drawn from the variational posterior distribution in the ELBO estimation process, and a single prediction is made.

Results Best-Case Noisy-Data Regression Models
The best-case models trained on the fully annotated noisy dataset were trained for 100 epochs. The models' predictive performances on the test set are reported. The best model in terms of RMSE was the BBB model. Thus, this model achieved RMSE of 0.832 at its best while the LeNet-300-100 model and the simple model were second-best with both their lowest RMSE being 0.835. The BBB model achieved its best predictive performance with fewer training iterations, i.e., at epoch 55, with the simple model reaching its best at 68 and the LeNet-300-100 model at 92. It is reiterated for the reader that a training iteration is much more computationally demanding for the BBB model, since the other models utilize closed form approximations for the purpose of reducing computational costs. See table 1 for a full report of evaluation metrics for the noisy data best-case models.

	Simple	LeNet-300-100	BBB
R2	0.296	0.300	**0.301**
MSE	0.697	0.693	**0.692**
RMSE	0.835	0.835	**0.832**
Adjusted R2	0.295	0.299	**0.300**
Achieved at epoch	68	92	**55**

Table 1: shows evaluation metrics for the regression models trained on the fully annotated noisy dataset. Metrics are reported and evaluated for the model checkpoint with the best achieved RMSE. The best score for each metric is highlighted in bold.

Further investigating Table 1, it becomes clear that the BBB model had the best R^2 score, indicating that this model was the best at correlating variance in the target variable *"quality of food purchased"* with the variance in the predictor variables. However, this is in close competition with the other models as the R^2 scores were 0.301 for the BBB model, 0.300 for the LeNet-300-100 model, and 0.296 for the simple model at the checkpoint with the lowest RMSE score. Looking at Figure 2, the differences in performance between the models were small, but the BBB model was better than the LeNet-300-100 model both in terms of R^2 and RMSE, while the LeNet-300-100 in turn was better than the simple model.

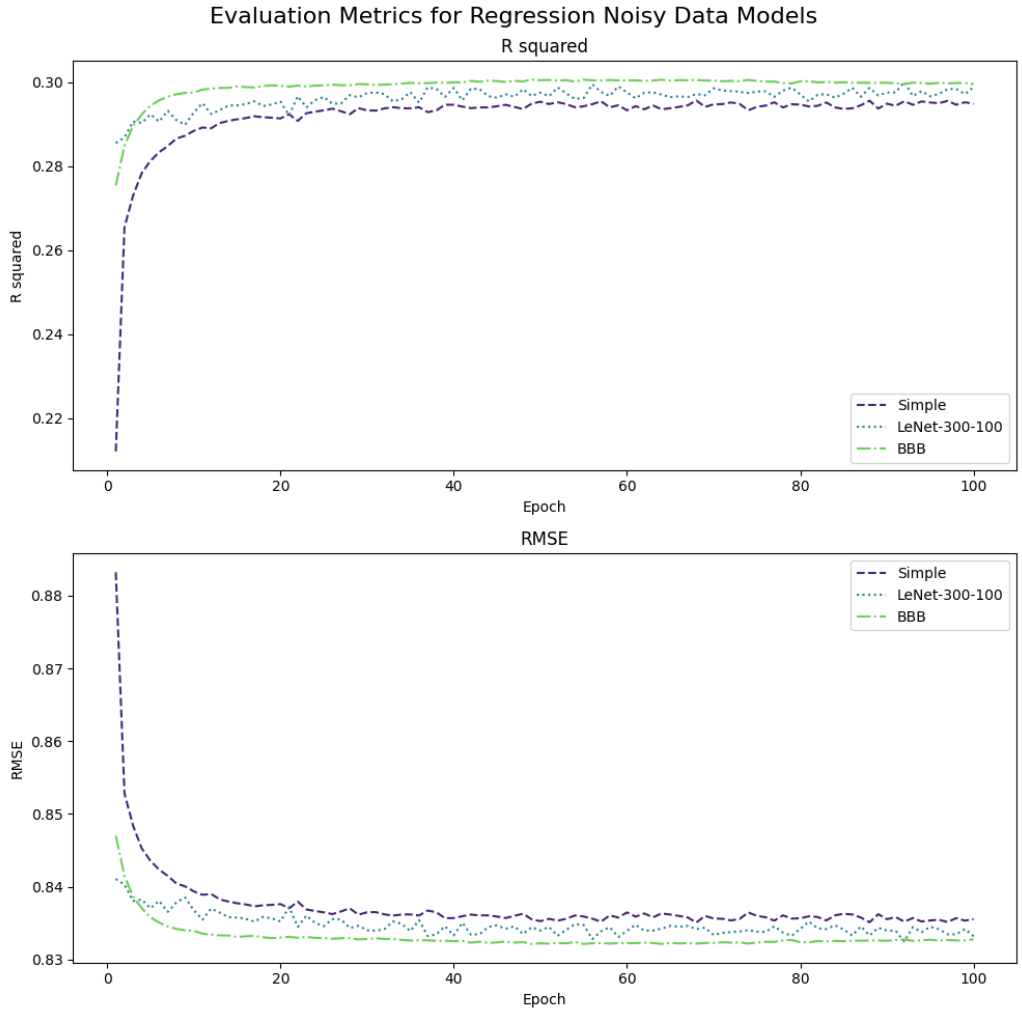


Figure 2: shows evaluation metrics for the different models trained on the regression noise data. The top panel shows R squared scores per epoch, and the bottom panel shows RMSE per epoch.

In sum, the regression models trained on the noisy data did not fit the data very well. Thus, the R^2 scores obtained for all the models indicate that a lot of the variance in the predictor variables cannot explain the variance in *"quality of food purchased"*. This is slightly contradictory to the actual correlations known to exist in the data. Looking at the correlation matrix in Figure 1 panel f, it is, e.g., known that the predictor *"monthly income"* in itself should explain 46% of the variance in *"quality of food purchased"*. However, the models do not fully learn this correlative pattern. The model that is best at learning the correlations between the predictors and the target was the BBB model and the model that was next best was the LeNet-300-100 model, while the simple model was the worst model.

Results Best-Case Relaxed-Data Regression Models
The best-case models trained on the fully annotated relaxed dataset were trained for 100 epochs. The models' predictive performances on a held-out test set are reported. The best model was the BBB model when evaluated on RMSE. Thus, this model reached an RMSE of 0.328 at its best, while the LeNet-300-100 model was second-best with its lowest RMSE being 0.329, and the worst model was the simple model with its lowest RMSE being 0.334. The BBB model achieved its best predictive performance with more training iterations, i.e., at epoch 100, with the simple model reaching its best at 94 and the LeNet-300-100 model at 95. See table 2 for a full report of evaluation metrics for the relaxed data best-case models. All in all, the models improved throughout almost all the training iterations. The BBB model reached its peak at the last epoch. However, the improvements were small for any iterations after epoch 30 and extremely small after 70 epochs for all the models. This is clearly seen in figure 3.

	Simple	LeNet-300-100	BBB
R2	0.889	**0.893**	**0.893**
MSE	0.112	**0.108**	**0.108**
RMSE	0.334	0.329	**0.328**
Adjusted R2	0.889	0.892	**0.893**
Achieved at epoch	**94**	95	100

Table 2: shows evaluation metrics for the regression models trained on the fully annotated relaxed dataset. Metrics are reported and evaluated for the model checkpoint with the best achieved RMSE. The best score for each metric is highlighted in bold.

Studying the contents of Table 2, the best model when evaluated on R^2 scores

was the BBB model with an R^2 of 0.893, whereas the second-best model was the LeNet-300-100 model with an R^2 of 0.892, and the worst model evaluated on this metric was the simple model with a score of 0.889. This was further confirmed by studying figure 3. This visualization shows that the BBB model and the LeNet-300-100 model were very similar in performance at all stages of the learning process, while the simple model was slightly worse. However, the models seem to follow the same learning pattern with a rapid improvement happening in the first iterations of learning and then a diminishing returns pattern occurring at the later stages of learning.

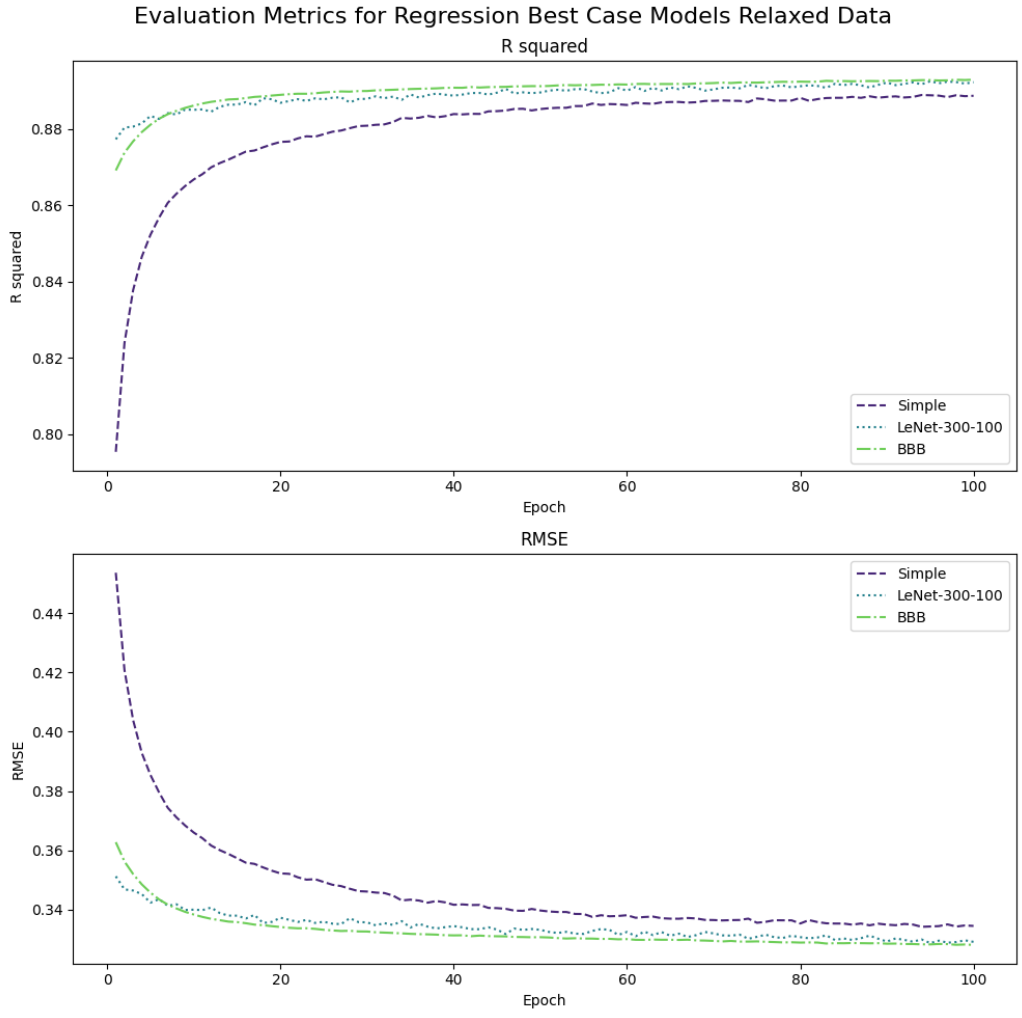


Figure 3: shows evaluation metrics for the different models trained on the regression relaxed data. The top panel shows R squared scores per epoch, and the bottom panel shows RMSE per epoch.

In conclusion, the regression models trained on the fully available relaxed data were extremely good at learning the patterns in the data. The BBB model was best for both the noisy and the relaxed data but in very close competition with the LeNet-300-100 model. All models found the same correlations in the data for both datasets, and the difference in performance between the models were small, although clear differences have been presented.

2.3.2 Results Best-Case Classification Models

Three classification architectures were trained on the classification datasets. The models were of a similar architecture as the regression models with an equivalent number of layers and hidden units for two closed form variational dropout architectures and for a Bayes by Backprop model. The models were trained on the classification datasets, which as a reminder to the reader, was identical to the regression data but for the fact that it was forced to have integer target variables, i.e., 1 through 5 with 5 being the best quality of food purchased. The classification models therefore implicitly necessitated a softmax activation function for their output layers.

For the best-case models trained on the fully annotated noisy dataset, some interesting findings showed. The three different models were trained for 100 epochs, and model performance was reported from the models' predictive abilities on the held-out test set. The best model in terms of accuracy was the LeNet-300-100 model. Thus, this model achieved an accuracy of 0.385 at its best with the BBB model being second-best with a peak accuracy score of 0.381, and the worst model was the simple model with its topmost accuracy being 0.370. However, it is worth noting that the BBB model achieved its best accuracy with fewer training iterations, i.e., at epoch 83, with the simple model reaching its best at 95 and LeNet-300-100 at 90. It is in this context reiterated that a training iteration is much more computationally demanding for the BBB model, since the other models utilize closed form approximations for the purpose of reducing computational costs. See table 3 for a full report of evaluation metrics for the noisy-data best-case models.

Results Noisy-Data Classification Models

	Simple	LeNet-300-100	BBB
Accuracy	0.370	**0.385**	0.381
F1	0.329	**0.363**	0.355
Class 1 F1	0.553	**0.564**	0.545
Class 2 F1	**0.272**	0.236	0.211
Class 3 F1	0.066	0.201	**0.268**
Class 4 F1	0.252	**0.313**	0.244
Class 5 F1	0.504	0.501	**0.505**
Achieved at epoch	95	90	**83**

Table 3: shows evaluation metrics for the classification models trained on the fully annotated relaxed dataset. Metrics are reported and evaluated for the model checkpoint with the best achieved accuracy. The best score for each metric is highlighted in bold.

Further examining table 3, it becomes clear that the Lenet-300-100 model achieved the best overall F1 score. However, this is in close competition with the BBB model with their overall F1 scores being 0.363 and 0.355 for the model checkpoint with the best accuracy. It is clear from figure 4 that the LeNet-300-100 and BBB models were better both in terms of accuracy and F1 scores at almost all points during the training process when compared with the simple model. The F1 score achieved for the simple model when accuracy was best was 0.329.

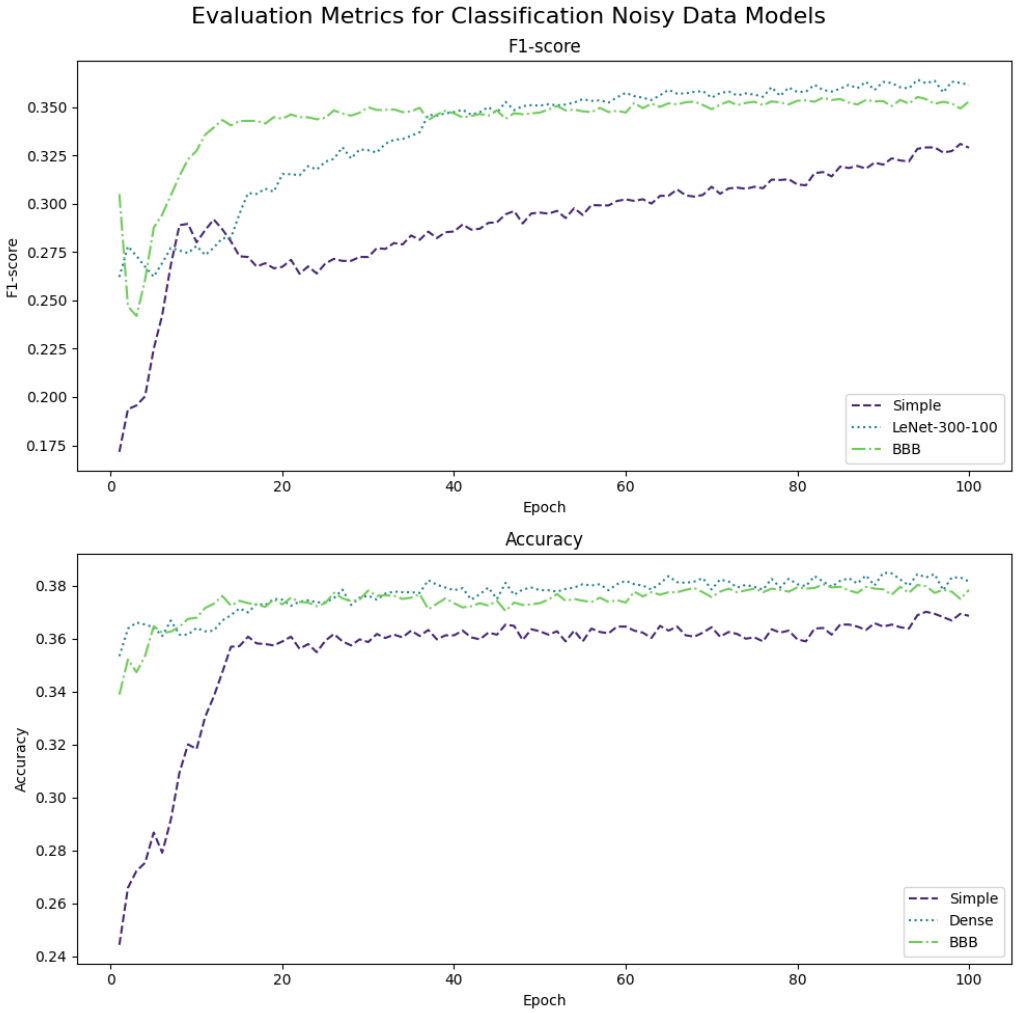


Figure 4: illustrates evaluation metrics for all the regression models trained on the annotated noisy data, i.e., the data synthesized with algorithm 1. The top panel shows F1, and the bottom panel shows accuracy. Both metrics are shown as a function of epochs. The simple closed form model is shown in purple, the LeNet-300-100 closed form model is shown in blue, and the BBB model is shown in green.

F1 per class is reported in table 3, and in figure 5 the models' F1 performance per class is visualized (see appendix 6 for tables with full per-class evaluation metrics report). From the visualization it becomes clear that the models performed best for the most extreme classes, i.e., class 1 and class 5. This is a recurring pattern in all the models tested on the noisy dataset. The models performed exceptionally bad for the middle classes 2, 3, and 4. The simple model was learning to classify the extreme classes at the cost of classifying the middle classes correctly. Thus, class 2 and 5 started off with relatively

high F1 scores in the early stages of the training process before trading the relatively good predictive performance of class 2 of with better performance of the extreme class 1. For both the LeNet-300-100 model and the BBB model the predictive performance stabilized for all the classes much earlier with the LeNet-300-100 model being very consistent with its performance levels after 40 iterations of training, and with the BBB model achieving stable predictive performance after just 15 epochs of training. The per class predictive performance therefore indicated that the BBB model fitted the data to its best potential at much fewer training iterations than the simple model. Furthermore, the LeNet-300-100 model's per-class predictive performance indicates that for more complex model architectures the closed-form models were faster at converging to its best predictive performance than for simple model architectures.

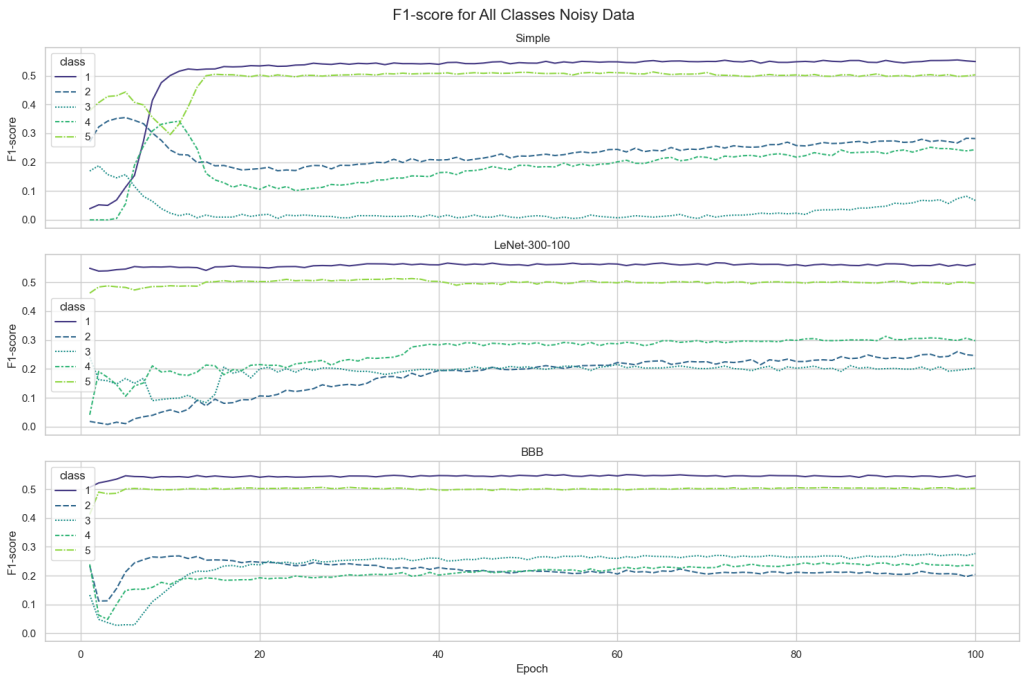


Figure 5: shows the F1 scores for each individual class in all the models trained on the noisy data. The top panel shows the simple closed form model. The middle panel shows the LeNet-300-100 model. The bottom panel shows the BBB model. All the panels show the F1 scores as a function of epochs in the training process.

In conclusion, all the classification models trained on the noisy dataset fitted the data poorly. The best model based on accuracy was the LeNet-300-100 model, and it achieved an accuracy of 0.385. Per class predictive ability was very similar for all the models with the models being best at predicting

extreme cases. However, the BBB model reached a stable predictive performance after fewer iterations of training than the LeNet-300-100 model and after much fewer iterations than the simple model.

Relaxed-Data Classification Models
The classification models trained on the relaxed dataset showed completely different predictive abilities compared to their counterpart models trained on the noisy dataset. The three different models were trained for 100 epochs, and model performance is reported from the models' predictive abilities on the held-out test set. The best model in terms of accuracy was the BBB model. Thus, this model achieved an accuracy of 0.995 at its best with the simple closed form and the LeNet-300-100 models being shared second-best with peak accuracies of 0.993. Similarly to the model trained on the noisy dataset, the BBB model achieved its best accuracy with fewer training iterations, i.e., at epoch 45, with the simple model reaching its best at 72 and LeNet-300-100 at 84. See table 4 for a full report of evaluation metrics for the noisy data best-case models.

	Simple	LeNet-300-100	BBB
Accuracy	0.993	0.993	**0.995**
F1	0.993	0.993	**0.995**
Class 1 F1	**0.998**	0.997	0.996
Class 2 F1	**0.995**	0.994	**0.995**
Class 3 F1	0.990	0.991	**0.994**
Class 4 F1	0.987	0.987	**0.993**
Class 5 F1	0.995	0.995	**0.999**
Achieved at epoch	72	84	**45**

Table 4: shows evaluation metrics for the classification models trained on the fully annotated noisy dataset. Metrics are reported and evaluated for the model checkpoint with the best achieved accuracy. The best score for each metric is highlighted in bold.

Further examining table 4, it becomes clear that the BBB model also achieved the best overall F1 score. However, this was in close competition with the other models with their overall F1 scores being 0.995 for the BBB model and 0.993 for the other models at their respective checkpoints with the best accuracy. It is clear from figure 6 that all the models had similar performance both when evaluated with accuracy and F1-scores. However, the BBB and simple models reached close to their best predictive performance after just 10 epochs, while the LeNet-300-100 model did so after 25 epochs. This indicates that the LeNet-300-100 model should be trained for more training iterations

than the other models to achieve its best predictive performance.

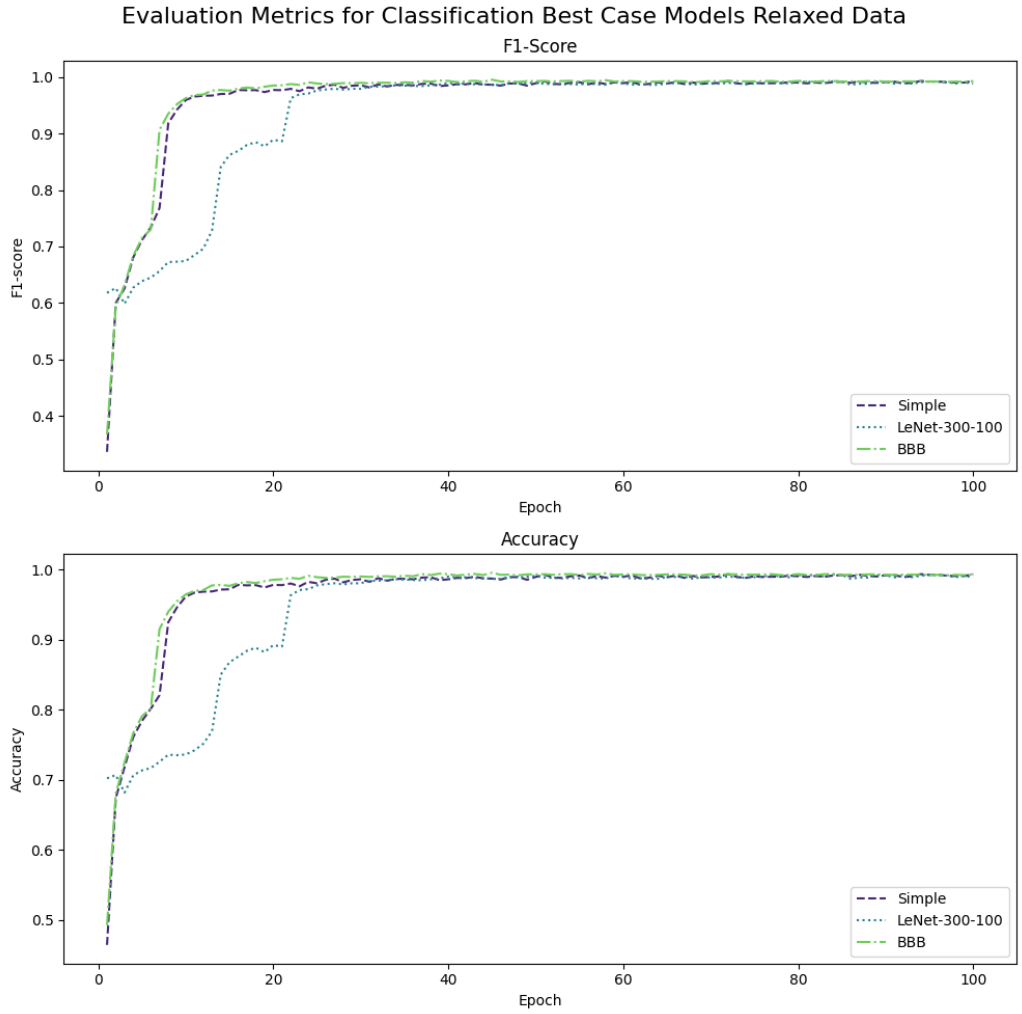


Figure 6: shows evaluation metrics for the three models trained on the best-case scenario with the relaxed data. I.e., the data has no noise and all the datapoints are annotated and available. The top panel shows F1 scores, and the bottom panel shows accuracy. Both metrics are shown as a function of epochs. The simple closed form model is shown in purple, the LeNet-300-100 closed form model is shown in blue, and the BBB model is shown in green. The best score for each metric is highlighted in bold.

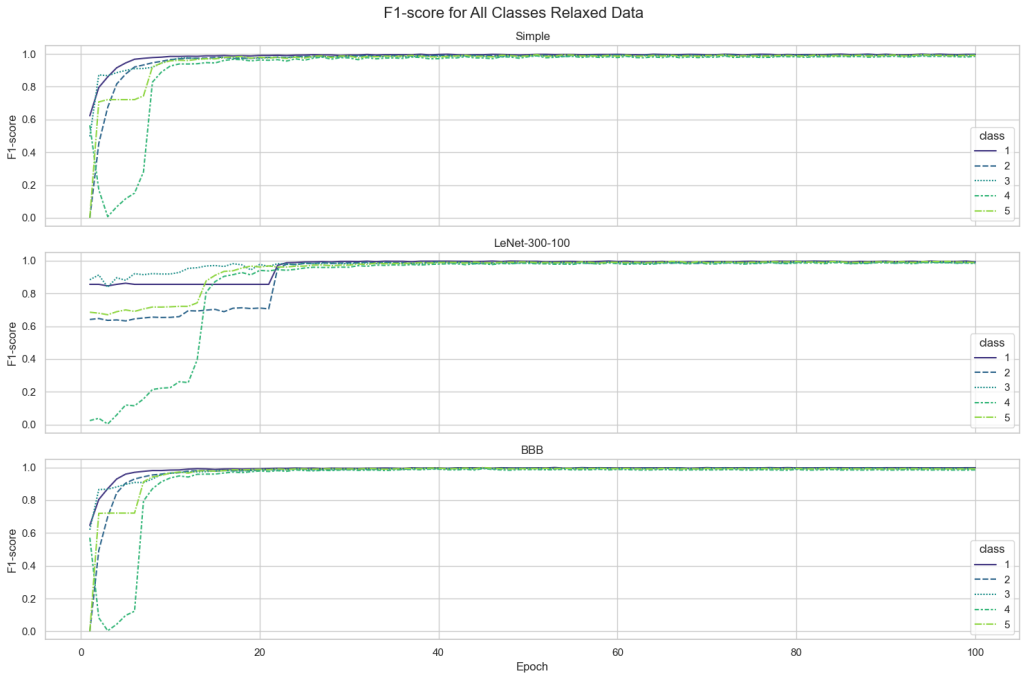


Figure 7: shows the F1 scores for each individual class in all the models trained on the relaxed data. The top panel shows the simple closed form model. The middle panel shows the LeNet-300-100 model. The bottom panel shows the BBB model. All the panels show the F1 scores as a function of epochs in the training process.

Figure 7 shows how the best-case classification models performed per class, and the models performed well on all classes with no visual differences in predictive performance for the fully trained models. However, all models use a higher number of training iterations to achieve good predictive performance of class 4. The pattern that was seen in the noisy data models with the extreme cases being the ones that were predicted best, was therefore not seen for the relaxed data. This suggests that the noisy models' superior predictive performances on the extreme classes could be due to artefacts in the noise and not due to the model architecture. In conclusion, the classification models trained on the relaxed dataset performed very close to perfectly, indicating that the models had great predictive abilities on unseen data, when the data had rigid correlations between predictor variables and the target variable. The BBB model had the best performance when evaluated on accuracy and F1 scores, and it reached its best predictive ability after much fewer training iterations than the other models.

2.4 Discussion of the Best-Case Models

The best-case models showed very different predictive abilities for differing levels of data complexity. The relaxed data was very easy to fit for the models which was hardly surprising when considering the simplicity behind its generative mechanism. However, the noisy data which was a far more realistic interpretation of human behavior was difficult to fit for the models (Darko et al., 2013; Pechey & Monsivais, 2015; Ricciuto et al., 2006; Turrell & Kavanagh, 2006). This could be due to several reasons; however, a big contributor could be argued to be the simplicity of the model architectures. All the models were implemented with only two hidden layers and a maximum of 300 nodes due to computational restrictions. Modern architectures can contain billions of parameters (H. Cheng et al., 2023). The VGG-16 network trained in Rakesh and Jain (2021), for example, contains 16 layers and around 138 million parameters. Extremely large networks might be better at capturing the nuances in complex datasets than simpler model architectures. For instance it has been shown that networks containing fewer than three hidden layers achieve poorer accuracy than larger networks because they are unable to learn the complexities in the training data (Uzair & Jamil, 2020). Another reason that the models were unable to get a decent grasp of the patterns in the data, could be a lack of patterns in the data. It is possible that the data generating mechanism was created too conservatively and that the correlations between the predictor variables and the target variable were too nebulous. *"guests"* e.g., only had a correlation of 0.14 with *"quality of food"*, and it is therefore quite possible that the true model of the data would also just have found a small effect. However, it should be noted that predictors such as *"savings"* and *"monthly income"* had stronger correlations with *"quality of food"*. It is therefore not unreasonable to expect the models to learn some sort of correlational pattern in the data, and looking at the classification models and the accuracies they achieved that was also somewhat the case. All models achieved accuracies over 0.37 on the noisy data, which is still a lot better than the 0.2 that would be expected had the models been made to guess randomly. Relating these findings with the fantastic performance of the models trained on the relaxed data, it is argued that BNNs can predict human behavior based of supplied predictor variables to some extent if the data is very complex and to a great degree if the underlying data generating mechanism is simple.

2.4.1 Choice of Prior

Another factor that could be influencing the models is the choice of prior. All models used a continuous relaxation of the spike-and-slab prior (Kingma et al., 2015; Louizos et al., 2017; Rakesh & Jain, 2021). This is a case of a sparsity inducing prior from the scale mixture family, and these are argued to be great for their sparsity inducing effect in the number of hidden units that need to be realized for every instance of a BNN (see sections about "Closed Form Bayesian Neural Network Layers Derived from Variational Dropout" and "dropout"), (Nguyen et al., 2021). However, studies have shown that the Laplace distribution might be a better choice of prior distributions for fully-connected BNNs (Foong et al., 2019; Fortuin et al., 2022). In fact, the Fortuin et al. (2022) study on the use of isotropic Gaussian priors in BNNs found that fully-connected neural networks had heavy-tailed weight distributions, and that heavy tailed priors were a better fit for fully-connected neural networks. Seeing as all the models implemented in this thesis were fully-connected neural networks, the choice of a continuous relaxation of the spike-and-slab prior is argued by the author of the thesis at hand to have been a proper choice of prior. To elaborate on this argument, consider the Fortuin et al. (2022) study and its critique of isotropic Gaussian priors, Fortuin et al. experimentally showed in this that fully-connected neural networks learn heavy tailed weight distributions and further that heavy-tailed specified priors improve the predictive performance of BNNs. When utilizing the continuous relaxation of the spike-and-slab prior, the prior is not isotropic, i.e., it has a spike element that enforces sparsity and a slab component that allows for flexibility. Essentially, the mixture variance for the slab component ensures that heavier tails are incorporated into the prior belief about weight distributions. In many ways the continuous relaxation of the spike-and-slab distribution achieves many of the same things that a Laplace distribution would, such as inducing sparsity with larger densities at zero and allowing for heavier tails while being flexible. However it is possible that better results might have been achieved with a Laplace slab, i.e., even larger tails than in the Gaussian spike-and-slab (Ray et al., 2020). All in all, the priors implemented in this thesis are argued to have been appropriately chosen and specified for the models tested in the experiments. This is argued in relation to the models having superior predictive performance on the relaxed dataset and showing some predictive abilities on the noisy dataset.

2.4.2 The Influence of the Specification of Model Hyperparameters

An additional specification that could have led to suboptimal predictive performance for the models in the thesis was the choice of sampling 3 times from the variational posterior distribution when trying to approximate the gradient of the ELBO in the BBB model. The choice was made, because each sample required a forward pass through the network and therefore was computationally expensive. However, using more samples could have shrunk the variance of the gradient estimates and potentially led to better convergence (Hønsi, 2023). However, when considering the consistent results between all the models across both noisy and relaxed datasets and regression and classification tasks, it is likely that an increase in samples from the variational posterior distribution would have generated no significant improvement in the BBB models' predictive performances. It is still worth considering if different model specifications could have significantly improved the models' predictive performance on unseen test data. Hyperparameters such as learning rate, choice of optimizer, number of instances, and batch size were chosen from a trial-and-error basis. Therefore, utilizing a grid search would have made for a more solid case that the models did not achieve lower than optimal predictive performance due to model misspecifications.

In conclusion, the models that were tested showed great predictive performance on the relaxed datasets, which was hardly surprising as this was a question of learning how the predictor *"montly income"* dictated the outcome *"quality of food purchased"* with no noise and only a small spurious correlation between the predictor *"guests"*. Nonetheless, they demonstrated that even small network architectures can be great at learning correlational patterns. Moreover, the performance of the models on the noisy datasets showed that the models could predict significantly above chance level, although their performance showed that noise in the data can greatly reduce model performance. For the next chapter of the thesis the models will be tested in an active learning pipeline on a fraction of the datasets. The models will be tested only on the relaxed data, although the regression models are tested on the noisy data and these results are reported in the appendix. The reasons for using the relaxed data in the active learning pipeline is twofold; firstly, to show how performance is influenced by having fewer data samples available on a dataset that is known to be within the models' capabilities to fit. Secondly, to show how the influence of a noisy oracle influences the models' predictive performance, i.e., how possibly faulty annotations influence the predictive performance as learning is done in rounds with accumulating

data samples.

3 Using BNNs for active learning

The aim of this chapter is to use BNNs in an active learning pipeline. The purpose is to demonstrate how using BNNs to provide uncertainty in deep learning predictions for data querying in an active learning pipeline is a desirable approach. If successful, this methodology has obvious applications for reducing costs involved in labelling expert domain datasets. This is because the active learning methodology proposes that reasonable predictive performance can be achieved with fewer data-samples than in traditional deep learning models (Song et al., 2019). Domain specific datasets and the difficulty of obtaining accurate labels for them is one of the main challenges of machine learning advances (Gu & Leroy, 2019). Therefore, finding a way to alleviate data labelling costs could have huge benefits for all fields relying on expert domain knowledge in the data labelling process. The BNN architectures used for active learning in this chapter are the same as in chapter 2. The models can therefore be compared to the best-case models trained on fully annotated datasets, and the consequences of having fewer datapoints available for training can be discussed.

3.1 Introduction of Active Learning and Oracle Annotaters

Active learning is a semi-supervised machine learning approach that queries an oracle-annotator with unlabeled data for annotation in iterations of data querying called "rounds" (Monarch, 2021). Often termed as part of "human-in-the-loop iterative machine learning", active learning has practical implementations in a wide range of tasks and is a cost-efficient way of acquiring annotated datasets for data that is expensive to label (B. Z. Li et al., 2020; Mosqueira-Rey et al., 2023; S. Eyke et al., 2020). In traditional active learning a seed model is trained on a limited subset of annotated data from a larger pool of data. This model is used for predictions on the remaining unlabeled data, and a specified number of the unlabeled data samples are actively chosen, or queried, for labeling and incorporation in the next iteration of learning. The datapoints that are queried for annotation are chosen using uncertainty estimates and the datapoints with the most predictive uncertainty are chosen. The idea behind this is that the most uncertain predictions arise for data points in regions of the output space that are hardest to predict. The subsequent sentiment is therefore that knowing the labels of these datapoints is very informative to the model's learning process and ultimately ability to predict the data (Ali et al., 2014; Hua et al., 2013). The iterative process of active learning is commonly 1) train a seed model,

2), use the seed model for prediction, 3), choose and annotate data points for the next round of learning, 4), retrain a new model from scratch with an updated and more informative dataset. However, approaches have been used in which the model is continually trained, not from scratch, but initialized with the weights it learned from the previous round of learning (Mundt et al., 2023; Rakesh & Jain, 2021). The critique of using this approach is that continually training models can make them prone to catastrophic forgetting where the models unlearn already learned patterns (Hu et al., 2019). For the remainder of the thesis "continual learning" shall refer to the act of actively querying datapoints for annotation in an iterative process where learning is the continued tuning of model parameters from previous rounds. Consequently, "active learning", is likeminded and in many cases used synonymously with continual learning. If specified, active learning can refer to the act of initializing model weights from scratch for each iterative round of learning.

A substantial challenge in active- and continual-learning is that it often relies on uncertainty estimates in predictions which traditional machine learning models do not supply. The uncertainty estimates supplied for BNN model predictions offer a solution to this challenge. Thus, BNNs have been used in uncertainty based continual learning (H. Li et al., 2021; Rakesh & Jain, 2021). In Ebrahimi et al. (2020) the authors adapt the learning rate in the continual training of their BNN models based on the uncertainty in the model parameter distributions. However, to the author's knowledge, no implementation of continual- or active- learning using BNNs has been made on real-world datasets, thus making the applicability claims of such pipelines meeker than they could be. This thesis contributes to already existing findings by applying BNNs to first a simulated dataset, next a real-world psychotherapy dataset.

3.1.1 Oracle Annotators

In active- and continual-learning the data points queried for annotation are annotated by oracles. Oracles are humans or machine annotators who are queried with the datapoints for labeling so they can be added to the labeled dataset for the next iteration of learning (Knaeble et al., 2020). In real-life applications one could imagine a machine learning task in which a doctor had annotated images of healthy lungs and images of lungs with illnesses, which is not far from actual studies carried out (Kumar & Kumar, 2022). In this scenario, annotation is costly and requires valuable time spent annotating for the expert, in this case the doctor, which could otherwise have been spent on patientcare. It is therefore desirable to require fewer annotated training

samples, and it is scenarios such as these the thesis at hand attempts to contribute valuable research towards.

Not all active learning oracles are humans, in some research machine programmed oracles have been utilized to emulate human expert annotating behavior (Chakraborty, 2020). These have been criticized for assuming that the oracle is infallible which is not reflective of human oracle behavior where multiple experts with different skill-levels are often used. Imagine that in the scenario with labeling of healthy and ill lungs three different doctors were put to the task of labeling the dataset and imagine that these doctors had three different degrees of experience. It is not an unrealistic scenario, and multiple studies have explored and argued for the use of noisy oracles (Gupta et al., 2019; Hoarau et al., 2022; Schubert et al., 2023).

3.1.2 Acquisition Functions

When the oracle is queried with new datapoints to annotate, the datapoints are selected by determining the uncertainty in the predictions with an acquisition method. Acquisition methods are functions or algorithms set in place to quantify uncertainty and thereby select the most informative datapoints. Various acquisition methods have been utilized, with the most widespread being the Shannon-entropy (Shannon, 1948). The Shannon entropy H finds the entropy for a discrete probability distribution P and supplies knowledge of the uncertainty in the predictions. For a classification BNN with C output classes, T forward passes with new weight realizations W_t from the trained posterior, will result in T number of vectors X_t with the given probabilities of each class c in C. Taking the mean of the probabilities of each class across all output vectors:

$$\hat{p}(y = c \mid x) = \frac{1}{T} \sum_{t} \hat{p}(y = c \mid x, W_t),$$

Where W_t is the tth realization of weights from the trained posterior. The probability estimates can be used to calculate the entropy with:

$$H(y \mid x) = -\sum_{c} \hat{p}(y = c \mid x) \log(p(y = c \mid x)) \tag{16}$$

Another widely used acquisition function is the variation ratio v that is simply the ration of predicted classes not in the modal predicted category to the modal predicted category:

$$v = 1 - \frac{f_m}{T} \tag{17}$$

Where f_m is the modal predicted category. Other acquisition methods have been presented, such as the BALD method (Beluch et al., 2018; Houlsby et al., 2011). However, this thesis will make use of only the Shannon entropy and the variation ratio for uncertainty quantification as these are the ones previous studies had the most success with (Beluch et al., 2018; Rakesh & Jain, 2021). Trial and error was used when training the models in the thesis at hand, and the variation ratio was used for regression models, while Shannon entropy was used for classification models.

3.1.3 Types of Uncertainty

Uncertainty has traditionally been divided into two different categories, i.e., aleatoric and epistemic (Hüllermeier & Waegeman, 2021). Aleatoric uncertainty is referring to randomness in the purest meaning of the word. That is; aleatoric uncertainty results from inherently random effects in the data. Contrastively, epistemic uncertainty is due to lack of knowledge of the best true model of the data. In this way epistemic uncertainty arises due to causes that can be reduced. Considering the models tested in the paper at hand, epistemic uncertainty would be uncertainty arising due to misspecifications of the model e.g., the true best model might consist of a more optimal combination of hidden layers, nodes, activation functions and specified hyperparameters. For this reason, epistemic uncertainty is also referred to as "reducible" uncertainty while aleatoric uncertainty is consequently called "irreducible" uncertainty. In the context of BNN model predictions, the uncertainty is a combination of the two types of uncertainty, i.e., some uncertainty may be explained by noise in the data while some could be reduced by training the model until it converges to the true best model. It is the latter that is attempted attained by incorporating actively chosen, new data into the training process.

3.1.4 Motivation for the Continual Learning Experiments

The chapter at hand seeks to test the usage of BNNs in a continual learning setting for a number of reasons. Firstly, very little research has been done on the topic and most of the research conducted has been done on benchmark datasets (Gal et al., 2017; Mohamadi & Amindavar, 2020; Rakesh & Jain, 2021; Woo, 2022). However, recently more attention has been on applying

BNN active learning to real-world datasets (Fronzi et al., 2021; Zhu et al., 2022). Secondly, to be able to discuss the expected performance dips in model predictive performance when comparing with the best-case models. Thirdly, to be transparent about the advantages and challenges connected to BNNs in continual learning before using BNNs on a real-world, not simulated dataset in chapter 4. The research already done on BNNs in active- or continual learning pipelines that has been presented in the thesis at hand all test their models without knowledge of a "best-case" model (Fronzi et al., 2021; Gal et al., 2017; Mohamadi & Amindavar, 2020; Rakesh & Jain, 2021; Woo, 2022; Zhu et al., 2022). Their research offers valuable insights into the applicability of BNNs, but they are hindered in their ability to ascribe the true effects of incorporating actively queried datapoints into their learning algorithms. This thesis offers a unique view into the true effects of annotating actively queried data points in a BNN continual learning pipeline. All in all, chapter 3 contributes new insight into existing scientific research on Bayesian deep learning used for active- and continual- learning.

3.2 Methods

Following the contributions of Rakesh and Jain (2021) in which BNNs are used for active learning, this chapter builds and attempts to incorporate their implementation of BNNs in an active learning process on the synthesized datasets introduced in chapter 2. In their work a seed model is trained and used for predictions on unlabeled data. Their framework utilizes the uncertainties for the predictions made by the seed model to actively choose a chosen number of datapoints to include in the next iteration of training. This logic is applied in an active learning pipeline for the models in the paper at hand. In contrast to the models made by Rakesh and Jain (2021), the models investigated in the current chapter were trained using synthetic data, which allows for a simple implementation of an oracle annotator in the labeling of the selected datapoints for the next iteration of the active learning process. In other words, it is possible to query the unlabeled datapoints returned by the seed models to a predefined function that labels them correctly. This predefined function is simply a matter of applying the same defined data generating mechanism as in the data generating process (see algorithm 1 in chapter 2). The active learning process thus applies many of the same components as the pipeline in Rakesh and Jain (2021), but in addition it incorporates an explicit oracle in the active data labeling process. The active learning was done by applying the following logic:

Algorithm 2: Active Data Selection and Training

Input: Training Dataset D_{train}, Test Dataset D_{test}, Seed Dataset D_s, Model M, Number of Rounds R, acquisition size k, Number of Instances T, Oracle Annotator A, *AcquisitionFunction*

Output: Trained Model M

Note: M is trained with the option of changing the hyperparameters to a desired combination of: optimizer, learning rate, instances, Round, AcquisitionFuction and batch size.

Function `ActiveDataSelectionAndTraining():`

- ensemble_outputs = [] ,list to store outputs of size T
- **for** $r \in R$ **do**
  - **if** $r == 1$ **then**
    - **for** *epoch* **do**
      - Train M using D_s
      - Test M using D_{test}
  - **else**
    - fetch most recent model M
    - **for** *epoch* **do**
      - Train M using D_s
    - **for** $X \in D_{train}$ **do**
      - $output_T \leftarrow M(X)$
      - ensemble_outputs.append($output_T$)
    - uncertainty_list = []
    - **for** *each* o_T *in ensemble_outputs* **do**
      - uncertainty_list.append(AcquisitionFunction(o_T))
    - uncertainty_list.sort_from_highest_to_lowest()
    - data_to_be_annotated $\leftarrow$ top k from uncertainty_list
    - annotated_data $\leftarrow$ Oracle(data_to_be_annotated), where Oracle applies the logic from Algorithm 1
    - $D_s \leftarrow D_s \cup$ annotated_data

Algorithm 2: shows dataselection and model training

3.2.1 Data

The data used to train the active learning models with, was the relaxed data from chapter 2. The reasons for choosing this data and not the noisy data were described at the end of chapter 2, but they boil down to knowing the data generating process and the models' ability to fit this while simulta-

neously being a demonstrator of model performance dip. Reasons for not choosing to use the models on the noisy dataset was that the results from chapter 2 suggested that the models were too simple, and the data was too complex to expect any results worthy of demonstrating the applicability of BNNs in active learning. However, models were run on the noisy dataset, and for transparency reasons active learning pipelines deploying BNNs are reported in appendix 7.2. The models were trained iteratively with the initial seed model counting 1000 labelled datapoints (a tenth of the data used to train the best-case models with). For each additional round 100 datapoints were queried for the oracle to label and added to the pool of annotated data.

The oracle that was queried with unlabeled data, was implemented as an extension of the data generating mechanism used to generate the noisy data with. This was a conscious choice to accommodate the notion of imperfect oracles as discussed in the introduction (Gupta et al., 2019; Hoarau et al., 2022; Schubert et al., 2023). The oracle was thus annotating by considering all the predictors as well as it incorporated noise into its annotations, albeit the data generating mechanism was absent of both noise and causality between any predictors and target but for *"monthly income"*. This was feasible as the causal relationship between the *"monthly income"* and *"food purchased"* was the same for the relaxed and noisy dataset, and the relationships between the other predictors offer a reallistic interpretation of how humans could be falsely biased when acting as oracles. I.e., the non-existent relationship between the other predictors is a way to induce noise in the annotations.

3.2.2 Continual Learning Regression Models

Regression models were run on the relaxed versions of the grocery shopping data. The models were run using the active learning pipeline described in algorithm 2. Firstly, a seed model was trained using 1000 annotated samples. Hereafter the models were trained in rounds. In each round the remaining unlabeled data was predicted with the most current model, and the 100 predictions with the most uncertainty were annotated with the oracle model and included in the next iteration of training. The models were trained for 10 rounds and for 100 epochs per round. The models were evaluated using the same metrics as the models in chapter 2. For the uncertainty estimation, 30 instances of the model were used to get distributional outputs. In other words, when predicting outputs 30 samples were drawn from the posterior weight distributions. The variation ratio described in the introduction was used to acquire these outputs as a single uncertainty estimate for each prediction.

3.2.3 Continual Learning Classification Models

Classification models were implemented in a similar manner to the regression models, with the natural difference that they were implemented on the relaxed data that was created for classification. Moreover, the Shannon entropy introduced in the introduction was used as the acquisition function to get uncertainty estimates for each prediction.

3.3 Results Continual Learning

3.3.1 Results Regression Models

Looking at table 5, the BBB model performed best with an RMSE score of 0.353. The next best model was the LeNet-300-100 model with its best achieved RMSE score being 0.355. Lastly, the simple model performed at its best with an RMSE of 0.376. The LeNet-300-100 and BBB models achieved their best scores already in round 3, while the simple model needed 10 rounds to achieve its best performance. This indicates that more complex network architectures learn patterns in non-complex data faster than simpler structures.

	Simple	LeNet-300-100	BBB
R2	0.858	0.873	**0.875**
MSE	0.141	0.126	**0.124**
RMSE	0.376	0.355	**0.353**
Adjusted R2	0.857	0.873	**0.874**
Achieved at epoch	10	**3**	**3**

Table 5: shows the evaluation metrics achieved for the round with the best achieved model for all the regression models. The metrics are reported for the round with the lowest RMSE score for all the regression models. The best score for each metric is highlighted in bold.

Furthermore, table 5 shows that the BBB and the LeNet-300-100 models were best in terms of R^2 as well. The BBB model was slightly better than the LeNet-300-100 model on this metric, with their scores being 0.875 and 0.873 respectively. The simple model reached an R^2 score of 0.858 for its best round. This further consolidates the finding that more complex models are better at learning patterns in non-noisy regression data. This was also confirmed after visual inspection of figure 8. It was clearly seen that the two larger model architectures learned their optimal states much earlier than

the simple model. The larger model architectures showed signs of plateauing after a couple of rounds of continual learning, while the simple model increased. This suggests that the simple model benefited more from larger amounts of available training data. Moreover, it is interesting to note that the LeNet-300-100 model had much better predictive abilities after the first round of continual learning than the BBB model before they reached the same predictive abilities after two rounds. This shows that in cases with less available data and limited computational power available, the LeNet-300-100 model should be the preferable choice of model.

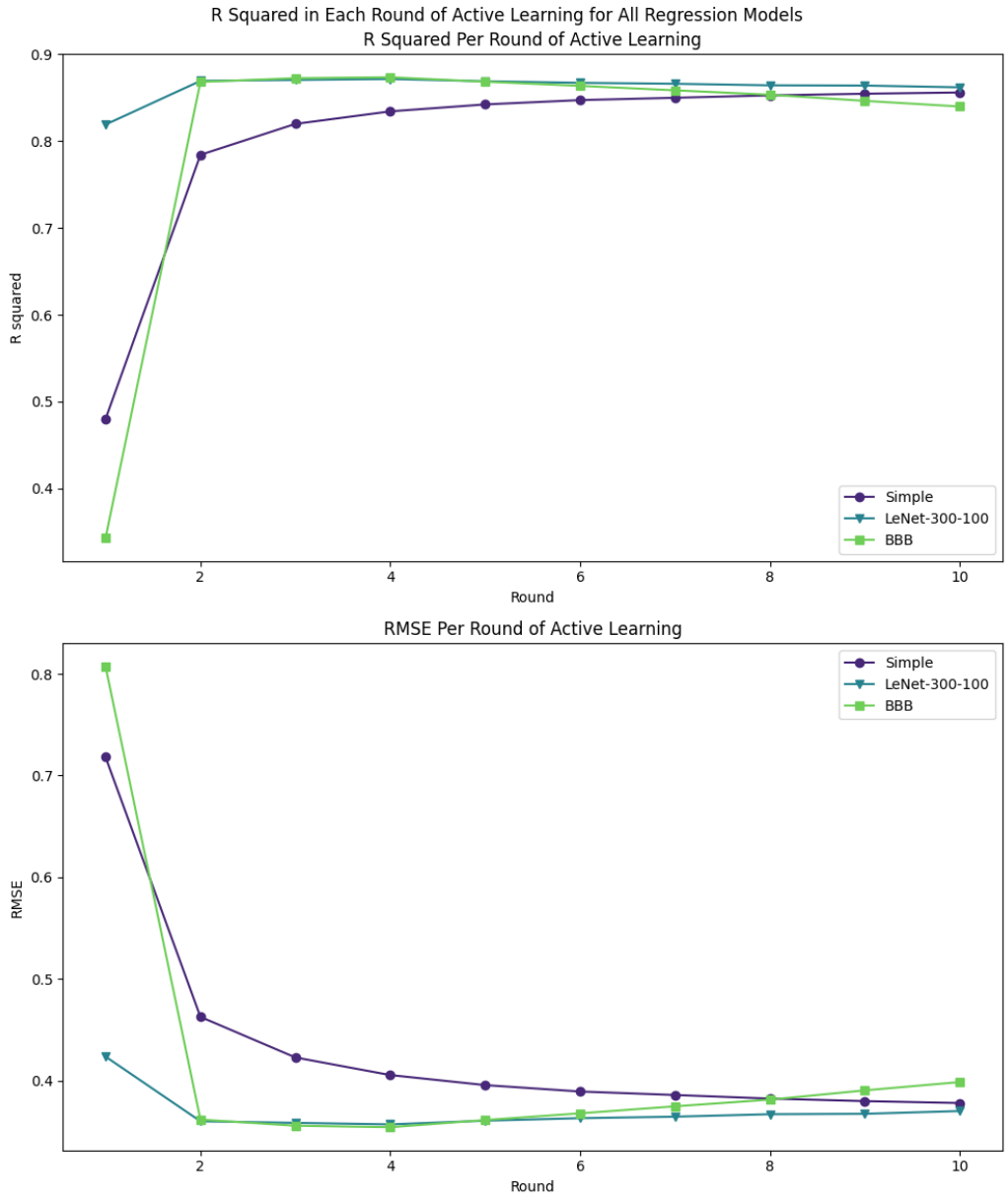


Figure 8: shows evaluation metrics per round of active learning for the regression models.

Relating the findings from this chapter to the previous chapter, the metrics should be compared for the best-case models and the continually trained models, for which the latter utilized far sparser training data. Due to the way RMSE is calculated the scores for the best-case models and the continually trained models are not comparable and the coefficient of determination R^2 is therefore used as a measure of comparison (Chicco et al., 2021). The simple

model trained on the relaxed data in chapter 2 achieved an R^2 of 0.889. Its continual learning counterpart reached a score of 0.858. The LeNet-300-100 best-case model achieved an R^2 of 0.893, and its equivalent continual learning model reached a score of 0.873. For the BBB model the best-case R^2 was 0.893, while the continually trained model reached a score of 0.875.

	Simple	LeNet-300-100	BBB
Absolute difference in R^2	0.031	0.020	**0.018**

Table 6: shows the absolute differences in R^2 between the best-case trained models and the continually trained models. The best score is highlighted in bold.

The absolute differences in R^2 between the best-case trained models and the continually trained models presented in table 6, inform about the cost it comes with to train the models with fewer data samples. The biggest deficit in predictive performance was demonstrated by the simple model that had an R^2 score that was 0.031 smaller than when all the annotated data was readily available for training. The second-biggest deficit was shown in the training of the LeNet-300-100 model which showcased a 0.020 drop in predictive performance when compared to its best-case counterpart. Lastly, the BBB model showed the best resistance to being trained with fewer data samples, as it only saw a 0.018 drop in R^2. These results show that it matters to the predictive performance of the models, whether they are trained with larger datasets even when almost no noise is present in the data.

All in all, the continually trained regression models showed great resistance to being trained in a continual learning setting with the predictive performance of the models seeing but a slight drop. The BBB model overall performed best but had only a slight advantage to the LeNet-300-100 model, which in turn showed better adaptability to being trained on fewer data samples when comparing the models after just a single round of training. The simple model showed less resistance to being trained on fewer data samples, and it took more iterations for this model to reach its peak than it did for the other models. In conclusion, the regression models all showed promising usability in a continual learning setting when trained on the relaxed data.

3.3.2 Results Classification Models

Looking at Table 7, the simple model performed best with an overall accuracy score of 0.940. The next best model was the LeNet-300-100 model with its best achieved accuracy score being 0.930. Lastly, the BBB model performed

at its best with an overall accuracy of 0.926. The LeNet-300-100 and simple models achieved their best scores in the last round of continual learning, while the BBB model only needed 6 rounds to achieve its best performance. This indicates that the closed form variational inference models benefitted more from adding continual learning rounds and newly annotated data to the training data. However, when looking at the visualization of the overall accuracy and F1 scores per round in Figure 9, similar learning patterns were revealed for the BBB model and the LeNet-300-100 model. Both came very close to their best predictive ability after 6 rounds of continual learning. After the sixth round, the predictive ability reached a plateau for both these models. Surprisingly, the simple model continued to improve throughout the rounds of continual learning. This suggests that simple model architectures are more susceptible to improvements when supplied with additional training data than larger model architectures in a continual learning classification task.

	Simple	LeNet-300-100	BBB
Accuracy	**0.940**	0.930	0.926
F1	**0.934**	0.924	0.917
Precision	**0.936**	0.925	0.920
Recall	**0.940**	0.935	0.927
Class 1 F1	**0.989**	0.987	0.**989**
Class 2 F1	0.953	0.938	**0.960**
Class 3 F1	**0.906**	0.890	0.902
Class 4 F1	0.872	0.851	0.812
Class 5 F1	0.952	**0.953**	0.924
Achieved in round	10	10	**6**

Table 7: shows the evaluation metrics achieved for the round with the best achieved model for all the classification models. The metrics are reported for the round with the lowest RMSE score for all the regression models. The best score for each metric is highlighted in bold.

Table 7 also reports per class F1 scores (refer to the appendix 7.1 for per class evaluation metrics). The models performed in the same order when evaluated on overall precision, recall and F1 as they did when evaluated on accuracy. This is also somewhat indicated from figure 9 where the F1 and accuracy scores follow similar patterns per round for the different models.

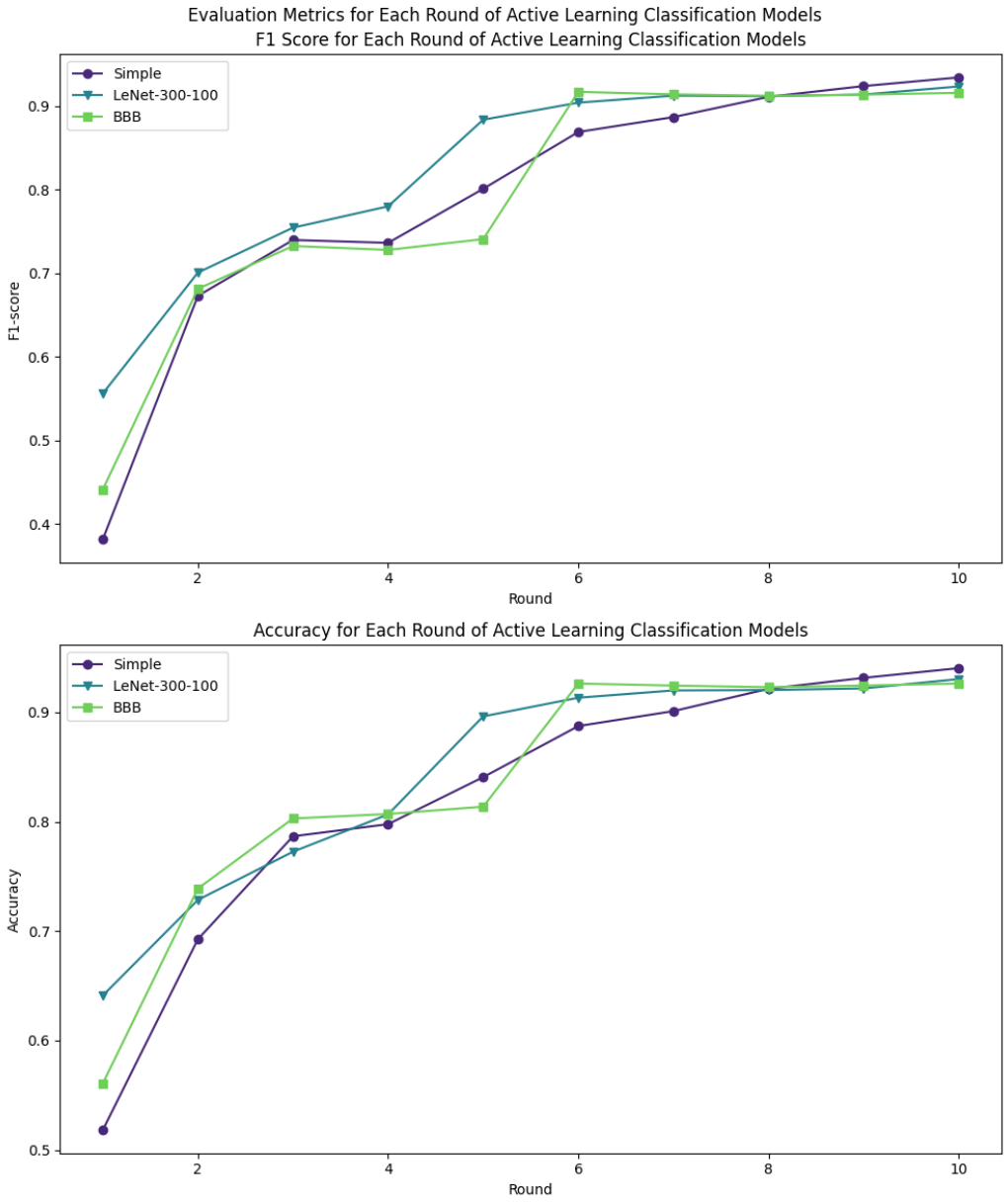


Figure 9: shows evaluation metrics for each round of active learning for the classification models.

When looking at per class differences between the models in figure 10, some interesting findings showed. All the models were clearly worse at predicting class 4, while both the simple and the LeNet-300-100 model found class 3 the second hardest to predict. All the models found class 1 the easiest to predict, which is a repeat pattern from their best-case counterpart models, however,

the BBB model predicted class 5 with the second worst F1 score. This might suggest that training in a continual learning setting could introduce some inaccuracies in model predictions on unlabeled data. This is especially likely when considering that the best-case BBB model showed no differences in per class predictive abilities. Both closed form models still displayed better predictive abilities for extreme data classes in the continual learning setting, however, this pattern was heavily dulled in comparison with the best-case models trained with the same architectures.

Relating the findings from this chapter to the previous chapter, the metrics are compared for the best-case models and the continually trained models, for which the latter utilized far sparser training data. For the classification models, the best-case and the continually trained models were compared on accuracy and F1 scores. The simple model trained on the relaxed data in chapter 2 achieved an accuracy of 0.993 and an F1 scores of 0.993. Its continual learning counterpart reached scores of 0.940 and 0.934. The LeNet-300-100 best-case model achieved scores of 0.993 and 0.993, and its equivalent continual learning model reached scores of 0.930 and 0.924. For the BBB model the best-case accuracy was 0.995 and F1 was 0.995, while the continually trained model reached scores of 0.926 and 0.917. In table 8 an overview over the absolute differences between the best-case trained models and the continually trained models is provided.

	Simple	LeNet-300-100	BBB
Accuracy Absolute Difference	**0.053**	0.063	0.069
F1 Absolute Difference	**0.059**	0.069	0.078

Table 8: reports absolute differences in predictive abilities for the models. Differences are between the best-case models trained on the fully available dataset and the model with the achieved accuracy in the continual learning scenario. The best score for each metric is highlighted in bold.

The absolute differences in accuracy and F1 between the best-case trained models and the continually trained models, inform about the cost it comes with to train the models with fewer data samples. The biggest deficit in predictive performance was demonstrated by the BBB model that had an accuracy score that was 0.069 smaller than when all the annotated data was readily available for training and an absolute difference of 0.078 in F1. The second-biggest deficit was shown in the testing of the LeNet-300-100 model which showcased a 0.063 drop in accuracy and a 0.069 drop in F1 when compared to its best-case counterpart. Lastly, the simple model showed the

best resistance to being trained with fewer data samples, as it only saw a 0.053 drop in accuracy and a 0.059 drop in F1. These results show that it matters to the predictive performance of the classification models, whether they are trained with larger datasets even when almost no noise is present in the data.

F1 Score for Each Class in Each Round of Active Learning for All Classification Models

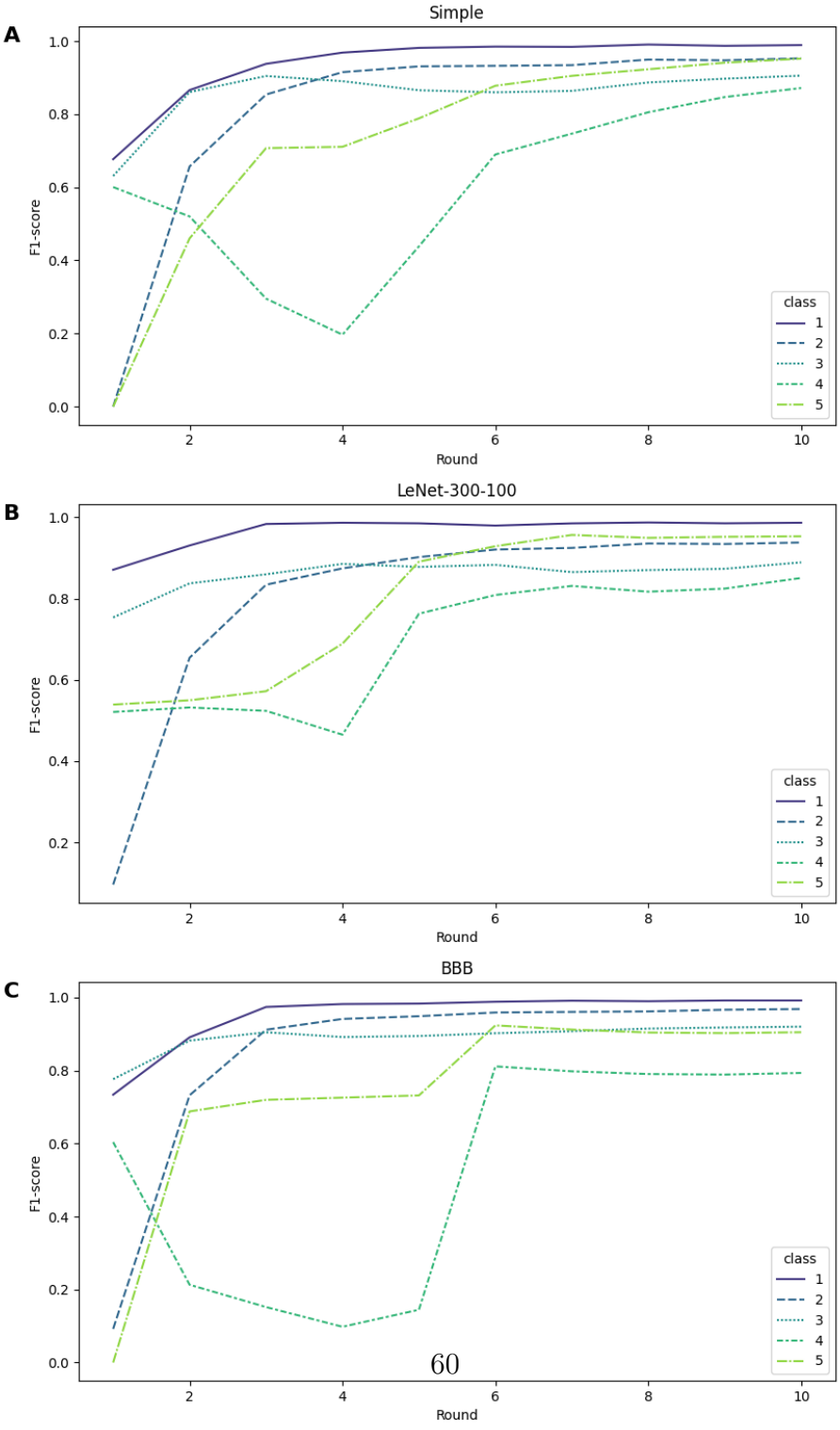


Figure 10: (figure on previous page) shows the per class F1 scores for the classification models. Panel A shows the per class scores for the simple model. Panel B shows the per class scores for the LeNet-300-100 model, and panel C shows the per class scores for the BBB model.

All in all, the continually trained classification models showed great resistance to being trained in a continual learning setting with the predictive performance of the models seeing but a slight drop. The simple model overall performed best and was clearly better at adapting to the continual learning setting than the LeNet-300-100 model which in turn had slightly less of a performance drop when compared to the BBB model. The models all followed quite similar learning paces in terms of which round they achieved their best score in; however, the BBB model reached its peak performance earlier than the other models. In conclusion all the classification models showed promising resistance to being trained on smaller datasets.

3.4 Discussion of Oracle Induced Noise and BNN Resistance to Noise in Continual Learning

The results for both the regression and classification models showed great resistance to active learning with imperfect oracles. It was evident from the experiments that the models' predictive performances never reached that of their best-case counterparts. Firstly, this is to be expected as training any form of machine-learning model on fewer datapoints would induce an expectation of a worse grasp of the causing patterns in the data (Vabalas et al., 2019). Secondly, the performance dip was quite small with all the classification models maintaining accuracy scores of above 0.92 and all regression models maintaining R^2 scores over 0.85. Thirdly the predictive performance is likely influenced by the noise inducing oracle.

Because the oracle was inducing both noise and annotating from a more complex generating algorithm than the data was originally produced with, the active learning rounds introduced noise into the noise-free relaxed-dataset as continual learning progressed. This is helpful in demonstrating that continual learning is robust against noise induced by imperfect oracles, even when trained on non-noisy datasets. Furthermore, it emulates a real-world scenario in which an experienced and skillful annotator labels the data for the seed model, and a less skillful annotator is responsible for being the oracle that is queried with unlabeled datapoints, which is a scenario not far from reality (Calma, 2020).

To convince to the reader why it is important to demonstrate the active learning pipeline's robustness to imperfect oracles, the reader is invited to

imagine the scenario given earlier in which three different doctors were tasked with annotating images of lung into categories of *"healthy"* and *"ill"*. This scenario implies that the annotations are intended to eventually be used to train a machine learning model to predict whether unseen images of lungs are *"healthy"* or *"ill"*, and ultimately to provide the best patientcare with the least cost. If this scenario is followed into an active learning pipeline as per the steps described in 3.1, a seed model would initially be trained on already annotated data. The seed model would then be used to predict on a larger pool of unseen images of lungs. The predictions would have uncertainty, and the predictions the seed model was most uncertain about would be given to the doctors so they could annotate them. This is all in the hopes that providing the model with more labelled data would allow it to improve its predictive performance. In the thought experiment, the three doctors have three different levels of expertise, and it is easy to imagine that the doctors would have different prerequisites for labelling the data. Consider, for example that the youngest doctor always classified images with dark spots as *"ill"*, when in fact dark spots is an artefact of the lung imaging technique used to obtain the images with. The young doctor's obliviousness to this fact would lead to wrongly labelled data, and ultimately bias the machine learning model to predict falsely as well. Humans are generally extremely fallible and prone to error, and showing that the models in this chapter are robust against some degree of artificially induced noise provides argumentative backing for the use of BNNs in an active- or continual-learning pipeline in a real-world context.

3.4.1 Contributions and Limitations

The work in this thesis contributes to existent literature on BNNs used for active learning and has brought forward a novel contribution to the influence of BNNs on real-world-like datasets with and without noise. The predictive performances of the models trained in the noisy data in the second chapter were very inconclusive, however, the degree of noise and randomness in the data generating mechanism held together with their well-above chance level performance, leaves a hopeful impression of the predictive performance of BNNs on a cognitive task dataset reflective of human behavior. When considering this with the very conclusive results from chapter 3 of this thesis, it is suggested that BNNs are a great alternative to other models for supplying uncertainty estimates for active learning. The thesis at hand is limited in its methodological approach as no alternative models to the BNNs were tested on the data. However, Rakesh and Jain (2021) demonstrated that BNNs are the superior choice for achieving quantifiable uncertainty estimates in an

active- and continual-learning setting. In the light of this finding, no other models were considered in the experiments in the thesis at hand. All in all, it is concluded that BNNs are the obvious choice for providing uncertainty for their predictions to be used for active learning.

3.4.2 Efficacy of Actively Queried Datapoints

The experiments in chapter 3 did not consider the degree to which improved model performance could be ascribed to the newly annotated data and how much was due to simply continually training for multiple rounds. It is possible that training the model on merely the seed data could have seen the model performance achieve similar predictive performance to that of the best-case models. However, the fact that the performance levels were only minimally negatively affected, is seen as grounds for suggesting that the learning was both aided by more available datapoints, and that noise induced by the imperfect oracles was not the sole reason for the minimally improved performance after few elapsed rounds of learning. The reader is also invited to look into appendix 7.2 in which regression models actively trained on noisy, imbalanced data are presented. Here it is demonstrated that the performance on noisy data and with noisy oracles sees absolute performance dips close to the ones reported for the regression models presented in chapter 3 for the first 10 rounds of learning. The reader should note that the presented actively trained regression models in appendix 7.2 were trained on a dataset not introduced anywhere else in this thesis and should therefore not be used for comparison.

The suggestion that the learning benefitted from queried datapoints is also grounded in previous research that explores the role of sample size in machine learning training (Ren et al., 2021). Even if it were the case that the models would have achieved learning equally fast without the queried datapoints, this would have merely shown that BNNs were models that were great at estimating the true model of the data generating mechanism. In this case the suggestion would still be to use BNNs in machine learning tasks solely due to their resistance to overfitting and ability to approximate the true model of a data generating mechanism. It has been shown that BNNs can be utilized for data querying oracles in active learning, and that using the oracle annotated datapoints only mildly reduces model predictive performance. Whether or not the performance dip is due to oracle induced noise or models having a harder time learning on smaller datasets is irrelevant for the point that is made in the thesis at hand, i.e., BNNs can be utilized in active learning pipelines, and using them does not catastrophically diminish model performance. In sum, it is suggested that BNNs would greatly benefit

active learning on cognitive datasets.

3.4.3 Future Work

In a paper on improved variational inference, authors describe "Pathfinder: Parallel quasi-Newton variational inference" (L. Zhang et al., 2022). Their approach returns draws from the posterior distribution with the lowest estimated KL divergence to the true posterior distribution. They compare their method to other alternatives and report improved results and parallelizability. Future implementations of BNNs could implement Pathfinder and investigate its impact. Furthermore, all the models in this thesis have utilized variational inference, but in the BNN community MCMC methods are also used for Bayesian inference (Chandra & Simmons, 2024). It would be exciting to implement MCMC alternatives to the models in the thesis at hand.

5 Conclusion

This thesis has introduced theoretical background for BNNs. Furthermore, it has implemented BNNs on synthesized datasets with and without noise in regression and classification tasks. Moreover, BNNs have been tested in a continual learning pipeline that compared model predictive ability with that of equivalent models trained on fully available datasets. Lastly, this thesis has shown proof-of-concept for how BNNs can be valauble models in the field of psychotherapy.

Chapter 1 provides a neat overview of the theoretical background of BNNs and understanding its contents is a necessary prerequisite for understanding the subsequent chapters. It is therefore included as a guidance into the world of BNN modeling.

Chapter 2 tested how 3 different model architectures, *simple*, *LeNet-300-100*, and *BBB* influenced model predictive performance on never seen before data. The data was synthesized in regression and classification versions and with and without noise. The data was emulating human grocery shopping behavior and thereby served as a demonstration into the fitting capabilities of BNNs on an easy-to follow experimental setup. Different conclusions were reached from the testing of BNN architectures on fully available datasets. Most notably it was shown that the models were able to find correlational patterns in both noisy and relaxed data. However, it was also shown that large amounts of irreducible noise leads to poorer model fits. The models fitted the relaxed data almost perfectly, however, the simple model was noticeably worse at fitting the noisy data in both regression and classification tasks, thus demonstrating how more complex network architectures are better equipped to learn complex data structures. It is concluded in this chapter that BNNs are able to learn complex data structures, and it is proposed that chapter 3 should show experiments conducted on the relaxed data.

Chapter 3 answered how BNNs can be used in active- and continual learning pipelines to alleviate labelling costs involved in obtaining annotated machine learning datasets. Models were trained continually on a tenth of the relaxed data introduced in chapter 2, and for every round of continual training another 100 samples were added to the pool of annotated data. For every round an imperfect oracle was queried. These experiments led to multiple conclusions. Firstly, the models generally improved for every new round of learning, however, the simple model kept increasing its predictive performance for each round of learning while the larger architectures decreased as noise was introduced by the oracle in the regression task. All the classification models increased their predictive performance for additional rounds of learning in the classification task. A discussion is provided about the effi-

cacy of actively querying datapoints for annotation, and it is concluded that BNNs offer a sound way to alleviate data labeling costs in hard-to-obtain datasets.

Chapter 4 demonstrated proof-of-concept of using BNNs in the field of psychotherapy. Autoregressive BNNs were used to validate other statistical measurements of dyadic HRV synchrony. No synchrony was found by neither the autoregressive BNN nor the other similarity tests. However, it is argued that BNNs offer a perfect way to validate other statistical methods in the field of psychotherapy due to their ability to fit complex data, and because of their inherent robustness against overfitting. Furthermore, a classification BNN was used in an active learning pipleine to show how BNNs could be used to find potential patterns between dyadic HRV synchrony and patient-therapist-alliance. The sample size was far to small to draw any conclusions from this experiment, but it is discussed how BNNs are the perfect tool to accomodate costly acquisition of data in the field of psychptherapy.

All in all, this thesis has demonstrated the applicability of BNNs in multiple scenarios. It has tested numerous different models, architectures and ways of training to answer the questions *what are the upsides, shortcomings, and use-cases of BNNs and how could they be used to alleviate data labelling costs involved in expert-domain dataset acquisition?*

Bibliography

Abdar, M., Pourpanah, F., Hussain, S., Rezazadegan, D., Liu, L., Ghavamzadeh, M., Fieguth, P., Cao, X., Khosravi, A., Acharya, U. R., Makarenkov, V., & Nahavandi, S. (2021). A review of uncertainty quantification in deep learning: Techniques, applications and challenges. *Information Fusion*, *76*, 243–297. https://doi.org/10.1016/j.inffus.2021.05.008

Abiodun, O. I., Jantan, A., Omolara, A. E., Dada, K. V., Mohamed, N. A., & Arshad, H. (2018). State-of-the-art in artificial neural network applications: A survey. *Heliyon*, *4*(11), e00938. https://doi.org/10.1016/j.heliyon.2018.e00938

Adamu, J. A. (2019). Advanced Stochastic Optimization Algorithm for Deep Learning Artificial Neural Networks in Banking and Finance Industries [Number: 1]. *Risk and Financial Management*, *1*(1), p8–p8. https://doi.org/10.30560/rfm.v1n1p8

Alfred Daniel, J., Chandru Vignesh, C., Muthu, B. A., Senthil Kumar, R., Sivaparthipan, C., & Marin, C. E. M. (2023). Fully convolutional neural networks for LIDAR–camera fusion for pedestrian detection in autonomous vehicle. *Multimedia Tools and Applications*, *82*(16), 25107–25130. https://doi.org/10.1007/s11042-023-14417-x

Ali, A., Caruana, R., & Kapoor, A. (2014). Active Learning with Model Selection [Number: 1]. *Proceedings of the AAAI Conference on Artificial Intelligence*, *28*(1). https://doi.org/10.1609/aaai.v28i1.9014

Atzil-Slonim, D., Soma, C. S., Zhang, X., Paz, A., & Imel, Z. E. (2023). Facilitating dyadic synchrony in psychotherapy sessions: Systematic review and meta-analysis [Publisher: Routledge _eprint: https://doi.org/10.1080/10503307.2023.2191803]. *Psychotherapy Research*, *33*(7), 898–917. https://doi.org/10.1080/10503307.2023.2191803

Bar-Kalifa, E., Prinz, J. N., Atzil-Slonim, D., Rubel, J. A., Lutz, W., & Rafaeli, E. (2019). Physiological synchrony and therapeutic alliance in an imagery-based treatment [Place: US Publisher: American Psychological Association]. *Journal of Counseling Psychology*, *66*(4), 508–517. https://doi.org/10.1037/cou0000358

Bayes, T. (1991). An essay towards solving a problem in the doctrine of chances. 1763. *M.D. Computing: Computers in Medical Practice*, *8*(3), 157–171.

Beluch, W. H., Genewein, T., Nürnberger, A., & Köhler, J. M. (2018). The Power of Ensembles for Active Learning in Image Classification, 9368–9377. Retrieved May 22, 2024, from https://openaccess.thecvf.com/content_cvpr_2018/html/Beluch_The_Power_of_CVPR_2018_paper.html

Binz, M., & Endres, D. (2019, February). Emulating Human Developmental Stages with Bayesian Neural Networks [arXiv:1902.07579 [cs, stat]]. https://doi.org/10.48550/arXiv.1902.07579

Blei, D. M., Kucukelbir, A., & McAuliffe, J. D. (2017). Variational Inference: A Review for Statisticians [Publisher: Taylor & Francis _eprint: https://doi.org/10.1080/01621459.2017.1285773]. *Journal of the American Statistical Association*, *112*(518), 859–877. https://doi.org/10.1080/01621459.2017.1285773

Blundell, C., Cornebise, J., Kavukcuoglu, K., & Wierstra, D. (2015). Weight Uncertainty in Neural Network [ISSN: 1938-7228]. *Proceedings of the 32nd International Conference on Machine Learning*, 1613–1622. Retrieved May 14, 2024, from https://proceedings.mlr.press/v37/blundell15.html

Bykov, K., Höhne, M. M.-C., Müller, K.-R., Nakajima, S., & Kloft, M. (2020, June). How Much Can I Trust You? – Quantifying Uncertainties in Explaining Neural Networks [arXiv:2006.09000 [cs, stat]]. https://doi.org/10.48550/arXiv.2006.09000

Calma, A. (2020, January). *Active Learning with Uncertain Annotators: Towards Dedicated Collaborative Interactive Learning* [Google-Books-ID: 4pIiEAAAQBAJ]. BoD – Books on Demand.

Chakraborty, S. (2020). Asking the Right Questions to the Right Users: Active Learning with Imperfect Oracles [Number: 04]. *Proceedings of the AAAI Conference on Artificial Intelligence*, *34*(04), 3365–3372. https://doi.org/10.1609/aaai.v34i04.5738

Chan, W.-S. (2022). On temporal aggregation of some nonlinear time-series models. *Econometrics and Statistics*, *21*, 38–49. https://doi.org/10.1016/j.ecosta.2020.03.008

Chandra, R., & Simmons, J. (2024). Bayesian neural networks via MCMC: A Python-based tutorial [Conference Name: IEEE Access]. *IEEE Access*, 1–1. https://doi.org/10.1109/ACCESS.2024.3401234

Cheng, H., Zhang, M., & Shi, J. Q. (2023, August). A Survey on Deep Neural Network Pruning-Taxonomy, Comparison, Analysis, and Recommendations [arXiv:2308.06767 [cs]]. https://doi.org/10.48550/arXiv.2308.06767

Cheng, Y.-C., Su, M.-I., Liu, C.-W., Huang, Y.-C., & Huang, W.-L. (2022). Heart rate variability in patients with anxiety disorders: A systematic review and meta-analysis [_eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/pcn.13356]. *Psychiatry and Clinical Neurosciences*, *76*(7), 292–302. https://doi.org/10.1111/pcn.13356

Chicco, D., Warrens, M. J., & Jurman, G. (2021). The coefficient of determination R-squared is more informative than SMAPE, MAE, MAPE, MSE and RMSE in regression analysis evaluation [Publisher: PeerJ Inc.]. *PeerJ Computer Science*, *7*, e623. https://doi.org/10.7717/peerj-cs.623

Coutinho, J., Pereira, A., Oliveira-Silva, P., Meier, D., Lourenço, V., & Tschacher, W. (2021). When our hearts beat together: Cardiac synchrony as an entry point to understand dyadic co-regulation in couples [_eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1111/psyp.13739]. *Psychophysiology*, *58*(3), e13739. https://doi.org/10.1111/psyp.13739

Darko, J., Eggett, D. L., & Richards, R. (2013). Shopping Behaviors of Low-income Families during a 1-Month Period of Time. *Journal of Nutrition Education and Behavior*, *45*(1), 20–29. https://doi.org/10.1016/j.jneb.2012.05.016

Dogo, E. M., Afolabi, O. J., Nwulu, N. I., Twala, B., & Aigbavboa, C. O. (2018). A Comparative Analysis of Gradient Descent-Based Optimization Algorithms on Convolutional Neural Networks. *2018 International Conference on Computational Techniques, Electronics and Mechanical Systems (CTEMS)*, 92–99. https://doi.org/10.1109/CTEMS.2018.8769211

Dusenberry, M. W., Tran, D., Choi, E., Kemp, J., Nixon, J., Jerfel, G., Heller, K., & Dai, A. M. (2020). Analyzing the Role of Model Uncertainty for Electronic Health Records [arXiv:1906.03842 [cs, stat]]. *Proceedings of the ACM Conference on Health, Inference, and Learning*, 204–213. https://doi.org/10.1145/3368555.3384457

Dynamic Time Warping. (2007). In M. Müller (Ed.), *Information Retrieval for Music and Motion* (pp. 69–84). Springer. https://doi.org/10.1007/978-3-540-74048-3_4

Ebrahimi, S., Elhoseiny, M., Darrell, T., & Rohrbach, M. (2020, February). Uncertainty-guided Continual Learning with Bayesian Neural Networks [arXiv:1906.02425 [cs, stat]]. https://doi.org/10.48550/arXiv.1906.02425

Esposito, P. (2024, March). piEsposito/blitz-bayesian-deep-learning [original-date: 2020-02-28T16:26:38Z]. Retrieved March 8, 2024, from https://github.com/piEsposito/blitz-bayesian-deep-learning

Federici, M., Ullrich, K., & Welling, M. (2017, December). Improved Bayesian Compression [arXiv:1711.06494 [stat]]. https://doi.org/10.48550/arXiv.1711.06494

Feldman, R. (2021). Social Behavior as a Transdiagnostic Marker of Resilience [Publisher: Annual Reviews]. *Annual Review of Clinical Psy-*

chology, *17*(Volume 17, 2021), 153–180. https://doi.org/10.1146/annurev-clinpsy-081219-102046

Fintz, M., Osadchy, M., & Hertz, U. (2022). Using deep learning to predict human decisions and using cognitive models to explain deep learning models [Publisher: Nature Publishing Group]. *Scientific Reports*, *12*(1), 4736. https://doi.org/10.1038/s41598-022-08863-0

Foong, A. Y. K., Li, Y., Hernández-Lobato, J. M., & Turner, R. E. (2019, June). 'In-Between' Uncertainty in Bayesian Neural Networks [arXiv:1906.11537 [cs, stat]]. https://doi.org/10.48550/arXiv.1906.11537

Fortuin, V., Garriga-Alonso, A., Ober, S. W., Wenzel, F., Rätsch, G., Turner, R. E., van der Wilk, M., & Aitchison, L. (2022, March). Bayesian Neural Network Priors Revisited [arXiv:2102.06571 [cs, stat]]. https://doi.org/10.48550/arXiv.2102.06571

Fronzi, M., Isayev, O., Winkler, D. A., Shapter, J. G., Ellis, A. V., Sherrell, P. C., Shepelin, N. A., Corletto, A., & Ford, M. J. (2021). Active Learning in Bayesian Neural Networks for Bandgap Predictions of Novel Van der Waals Heterostructures [_eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/aisy.202100080]. *Advanced Intelligent Systems*, *3*(11), 2100080. https://doi.org/10.1002/aisy.202100080

Gal, Y., & Ghahramani, Z. (2016). Dropout as a Bayesian Approximation: Representing Model Uncertainty in Deep Learning [ISSN: 1938-7228]. *Proceedings of The 33rd International Conference on Machine Learning*, 1050–1059. Retrieved May 13, 2024, from https://proceedings.mlr.press/v48/gal16.html

Gal, Y., Islam, R., & Ghahramani, Z. (2017). Deep Bayesian Active Learning with Image Data [ISSN: 2640-3498]. *Proceedings of the 34th International Conference on Machine Learning*, 1183–1192. Retrieved May 21, 2024, from https://proceedings.mlr.press/v70/gal17a.html

Gawlikowski, J., Tassi, C. R. N., Ali, M., Lee, J., Humt, M., Feng, J., Kruspe, A., Triebel, R., Jung, P., Roscher, R., Shahzad, M., Yang, W., Bamler, R., & Zhu, X. X. (2022, January). A Survey of Uncertainty in Deep Neural Networks [arXiv:2107.03342 [cs, stat]]. https://doi.org/10.48550/arXiv.2107.03342

Gawlikowski, J., Tassi, C. R. N., Ali, M., Lee, J., Humt, M., Feng, J., Kruspe, A., Triebel, R., Jung, P., Roscher, R., Shahzad, M., Yang, W., Bamler, R., & Zhu, X. X. (2023). A survey of uncertainty in deep neural networks. *Artificial Intelligence Review*, *56*(1), 1513–1589. https://doi.org/10.1007/s10462-023-10562-9

Giorgino, T. (2009). Computing and Visualizing Dynamic Time Warping Alignments in R: The dtw Package. *Journal of Statistical Software*, *31*, 1–24. https://doi.org/10.18637/jss.v031.i07

Gu, Y., & Leroy, G. (2019). Mechanisms for automatic training data labeling for machine learning. Retrieved April 11, 2024, from https://scholar.archive.org/work/zsnmpzg5tnh7bp7vwn3epwvpgy/access/wayback/https://aisel.aisnet.org/cgi/viewcontent.cgi?article=1044&context=icis2019

Guerra, J. L., Catania, C., & Veas, E. (2022). Datasets are not enough: Challenges in labeling network traffic. *Computers & Security*, *120*, 102810. https://doi.org/10.1016/j.cose.2022.102810

Gupta, G., Sahu, A. K., & Lin, W.-Y. (2019). Learning in Confusion: Batch Active Learning with Noisy Oracle. Retrieved May 21, 2024, from https://openreview.net/forum?id=SJxIkkSKwB

Han, R., Gill, M., Spirling, A., & Cho, K. (2018, October). Conditional Word Embedding and Hypothesis Testing via Bayes-by-Backprop. In E. Riloff, D. Chiang, J. Hockenmaier, & J. Tsujii (Eds.), *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing* (pp. 4890–4895). Association for Computational Linguistics. https://doi.org/10.18653/v1/D18-1527

Hartmann, R., Schmidt, F. M., Sander, C., & Hegerl, U. (2019). Heart Rate Variability as Indicator of Clinical State in Depression [Publisher: Frontiers]. *Frontiers in Psychiatry*, *9*. https://doi.org/10.3389/fpsyt.2018.00735

Hatcher, R. L., & Gillaspy, J. A. (2006). Development and validation of a revised short version of the working alliance inventory [Publisher: Routledge
_eprint: https://doi.org/10.1080/10503300500352500]. *Psychotherapy Research*, *16*(1), 12–25. https://doi.org/10.1080/10503300500352500

Hinton, G. E., Srivastava, N., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. R. (2012, July). Improving neural networks by preventing co-adaptation of feature detectors [arXiv:1207.0580 [cs]]. https://doi.org/10.48550/arXiv.1207.0580

Hoarau, A., Martin, A., Dubois, J.-C., & Le Gall, Y. (2022). Imperfect Labels with Belief Functions for Active Learning. In S. Le Hégarat-Mascle, I. Bloch, & E. Aldea (Eds.), *Belief Functions: Theory and Applications* (pp. 44–53). Springer International Publishing. https://doi.org/10.1007/978-3-031-17801-6_5

Hønsi, A. (2023, October). *Gaussian Likelihoods in Bayesian Neural Networks* [Master's thesis, The University of Bergen] [Accepted: 2023-11-

09T00:42:02Z]. Retrieved January 13, 2024, from https://bora.uib.no/bora-xmlui/handle/11250/3101521

Horvath, A. O., & Greenberg, L. S. (1989). Development and validation of the Working Alliance Inventory [Place: US Publisher: American Psychological Association]. *Journal of Counseling Psychology*, *36*(2), 223–233. https://doi.org/10.1037/0022-0167.36.2.223

Houlsby, N., Huszár, F., Ghahramani, Z., & Lengyel, M. (2011, December). Bayesian Active Learning for Classification and Preference Learning [arXiv:1112.5745 [cs, stat]]. https://doi.org/10.48550/arXiv.1112.5745

Hu, W., Lin, Z., Liu, B., Tao, C., Tao, Z. T., Zhao, D., Ma, J., & Yan, R. (2019). OVERCOMING CATASTROPHIC FORGETTING FOR CONTINUAL LEARNING VIA MODEL ADAPTATION. *International Conference on Learning Representations*. Retrieved May 21, 2024, from %5Cnewline%20https://par.nsf.gov/biblio/10120468-overcoming-catastrophic-forgetting-continual-learning-via-model-adaptation

Hua, G., Long, C., Yang, M., & Gao, Y. (2013). Collaborative Active Learning of a Kernel Machine Ensemble for Recognition, 1209–1216. Retrieved May 21, 2024, from https://openaccess.thecvf.com/content_iccv_2013/html/Hua_Collaborative_Active_Learning_2013_ICCV_paper.html

Hüllermeier, E., & Waegeman, W. (2021). Aleatoric and epistemic uncertainty in machine learning: An introduction to concepts and methods. *Machine Learning*, *110*(3), 457–506. https://doi.org/10.1007/s10994-021-05946-3

Hunter, D., Yu, H., Pukish III, M. S., Kolbusz, J., & Wilamowski, B. M. (2012). Selection of proper neural network sizes and architectures—A comparative study [Publisher: IEEE]. *IEEE Transactions on Industrial Informatics*, *8*(2), 228–240. Retrieved May 16, 2024, from https://ieeexplore.ieee.org/abstract/document/6152147/

Izmailov, P., Vikram, S., Hoffman, M. D., & Wilson, A. G. G. (2021). What Are Bayesian Neural Network Posteriors Really Like? [ISSN: 2640-3498]. *Proceedings of the 38th International Conference on Machine Learning*, 4629–4640. Retrieved October 26, 2023, from https://proceedings.mlr.press/v139/izmailov21a.html

Jia, S., Yue, Y., Yang, Z., Pei, X., & Wang, Y. (2020). Travelling Modes Recognition via Bayes Neural Network with Bayes by Backprop Algorithm [Publisher: American Society of Civil Engineers], 3994–4004. https://doi.org/10.1061/9780784482933.343

Jordon, J., Wilson, A., & van der Schaar, M. (2020, December). Synthetic Data: Opening the data floodgates to enable faster, more directed development of machine learning methods [arXiv:2012.04580 [cs]]. https://doi.org/10.48550/arXiv.2012.04580

Jung, Y. (2018). Multiple predicting K-fold cross-validation for model selection [Publisher: Taylor & Francis _eprint: https://doi.org/10.1080/10485252.2017.1404598]. *Journal of Nonparametric Statistics*, *30*(1), 197–215. https://doi.org/10.1080/10485252.2017.1404598

Kingma, D. P., & Welling, M. (2013, December). Auto-Encoding Variational Bayes [arXiv:1312.6114 [cs, stat] version: 1]. https://doi.org/10.48550/arXiv.1312.6114

Kingma, D. P., Salimans, T., & Welling, M. (2015). Variational Dropout and the Local Reparameterization Trick. *Advances in Neural Information Processing Systems*, *28*. Retrieved March 26, 2024, from https://proceedings.neurips.cc/paper/2015/hash/%20%5Cnewline%20bc7316929fe1545bf0b98d114ee3ecb8-Abstract.html

Knaeble, M., Nadj, M., & Maedche, A. (2020). Oracle or Teacher? A Systematic Overview of Research on Interactive Labeling for Machine Learning. *Wirtschaftsinformatik (Zentrale Tracks)*, 2–16. Retrieved May 21, 2024, from https://www.researchgate.net/profile/Merlin-Knaeble-2/publication/339797494_Oracle_or_Teacher_A_Systematic_Overview_of_Research_on_Interactive_Labeling_for_Machine_Learning/links/5eda5618299bf1c67d41d30e/Oracle-or-Teacher-A-Systematic-Overview-of-Research-on-Interactive-Labeling-for-Machine-Learning.pdf

Kocadağlı, O., & Aşıkgil, B. (2014). Nonlinear time series forecasting with Bayesian neural networks. *Expert Systems with Applications*, *41*(15), 6596–6610. https://doi.org/10.1016/j.eswa.2014.04.035

Kumar, S., & Kumar, H. (2022). Lungcov: A diagnostic framework using machine learning and Imaging Modality. *International Journal on "Technical and Physical Problems of Engineering"(IJTPE)*, *14*(51), 190–9. Retrieved May 21, 2024, from http://www.iotpe.com/IJTPE/IJTPE-2022/IJTPE-Issue51-Vol14-No2-Jun2022/23-IJTPE-Issue51-Vol14-No2-Jun2022-pp190-199.pdf

Kwon, Y., Won, J.-H., Kim, B. J., & Paik, M. C. (2020). Uncertainty quantification using Bayesian neural networks in classification: Application to biomedical image segmentation. *Computational Statistics & Data Analysis*, *142*, 106816. https://doi.org/10.1016/j.csda.2019.106816

Lakshminarayanan, B., Pritzel, A., & Blundell, C. (2017). Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles. *Ad-*

vances in Neural Information Processing Systems, *30*. Retrieved May 13, 2024, from https://proceedings.neurips.cc/paper_files/paper/2017/hash/9ef2ed4b7fd2c810847ffa5fa85bce38-Abstract.html

Lecun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition [Conference Name: Proceedings of the IEEE]. *Proceedings of the IEEE*, *86*(11), 2278–2324. https://doi.org/10.1109/5.726791

Li, B. Z., Stanovsky, G., & Zettlemoyer, L. (2020, May). Active Learning for Coreference Resolution using Discrete Annotation [arXiv:2004.13671 [cs]]. https://doi.org/10.48550/arXiv.2004.13671

Li, H., Barnaghi, P., Enshaeifar, S., & Ganz, F. (2021). Continual Learning Using Bayesian Neural Networks [Conference Name: IEEE Transactions on Neural Networks and Learning Systems]. *IEEE Transactions on Neural Networks and Learning Systems*, *32*(9), 4243–4252. https://doi.org/10.1109/TNNLS.2020.3017292

Liu, B., Wei, Y., Zhang, Y., & Yang, Q. (2017). Deep Neural Networks for High Dimension, Low Sample Size Data. *IJCAI*, *2017*, 2287–2293. Retrieved May 29, 2024, from https://www.ijcai.org/Proceedings/2017/0318.pdf

Liu, J., Paisley, J., Kioumourtzoglou, M.-A., & Coull, B. (2019). Accurate Uncertainty Estimation and Decomposition in Ensemble Learning. *Advances in Neural Information Processing Systems*, *32*. Retrieved May 13, 2024, from https://proceedings.neurips.cc/paper/2019/hash/1cc8a8ea51cd0adddf5dab504a285915-Abstract.html

Liu, M., Chen, L., Du, X., Jin, L., & Shang, M. (2023). Activated Gradients for Deep Neural Networks [Conference Name: IEEE Transactions on Neural Networks and Learning Systems]. *IEEE Transactions on Neural Networks and Learning Systems*, *34*(4), 2156–2168. https://doi.org/10.1109/TNNLS.2021.3106044

Liu, Y., Dong, W., Zhang, L., Gong, D., & Shi, Q. (2019). Variational Bayesian Dropout With a Hierarchical Prior, 7124–7133. Retrieved April 2, 2024, from https://openaccess.thecvf.com/content_CVPR_2019/html/Liu_Variational_Bayesian_Dropout_With_a_Hierarchical_Prior_CVPR_2019_paper.html

Louizos, C., Ullrich, K., & Welling, M. (2017). Bayesian Compression for Deep Learning. *Advances in Neural Information Processing Systems*, *30*. Retrieved January 17, 2024, from https://proceedings.neurips.cc/paper_files/paper/2017/hash/69d1fc78dbda242c43ad6590368912d4-Abstract.html

Mather, M., & Thayer, J. F. (2018). How heart rate variability affects emotion regulation brain networks. *Current Opinion in Behavioral Sciences*, *19*, 98–104. https://doi.org/10.1016/j.cobeha.2017.12.017

McDermott, P. L., & Wikle, C. K. (2019). Bayesian Recurrent Neural Network Models for Forecasting and Quantifying Uncertainty in Spatial-Temporal Data [Number: 2 Publisher: Multidisciplinary Digital Publishing Institute]. *Entropy*, *21*(2), 184. https://doi.org/10.3390/e21020184

Milstein, N., & Gordon, I. (2020). Validating Measures of Electrodermal Activity and Heart Rate Variability Derived From the Empatica E4 Utilized in Research Settings That Involve Interactive Dyadic States [Publisher: Frontiers]. *Frontiers in Behavioral Neuroscience*, *14*. https://doi.org/10.3389/fnbeh.2020.00148

Mitchell, T. J., & Beauchamp, J. J. (1988). Bayesian Variable Selection in Linear Regression [Publisher: Taylor & Francis _eprint: https://www.tandfonline.com/doi/pdf/10.1080/01621459.1988.10478694]. *Journal of the American Statistical Association*, *83*(404), 1023–1032. https://doi.org/10.1080/01621459.1988.10478694

Mitros, J., & Mac Namee, B. (2019, December). On the Validity of Bayesian Neural Networks for Uncertainty Estimation [arXiv:1912.01530 [cs, stat]]. https://doi.org/10.48550/arXiv.1912.01530

Mohamadi, S., & Amindavar, H. (2020, December). Deep Bayesian Active Learning, A Brief Survey on Recent Advances [Publication Title: arXiv e-prints ADS Bibcode: 2020arXiv201208044M]. https://doi.org/10.48550/arXiv.2012.08044

Molchanov, D., Ashukha, A., & Vetrov, D. (2017). Variational Dropout Sparsifies Deep Neural Networks [ISSN: 2640-3498]. *Proceedings of the 34th International Conference on Machine Learning*, 2498–2507. Retrieved February 1, 2024, from https://proceedings.mlr.press/v70/molchanov17a.html

Monarch, R. ((2021, August). *Human-in-the-Loop Machine Learning: Active learning and annotation for human-centered AI* [Google-Books-ID: bNo2EAAAQBAJ]. Simon; Schuster.

Mosqueira-Rey, E., Hernández-Pereira, E., Alonso-Ríos, D., Bobes-Bascarán, J., & Fernández-Leal, Á. (2023). Human-in-the-loop machine learning: A state of the art. *Artificial Intelligence Review*, *56*(4), 3005–3054. https://doi.org/10.1007/s10462-022-10246-w

Mostafa, B., Hassan, R., Mohammed, H., & Tawfik, M. (2023). A Review of Variational Inference for Bayesian Neural Network. In T. Masrour,

H. Ramchoun, T. Hajji, & M. Hosni (Eds.), *Artificial Intelligence and Industrial Applications* (pp. 231–243). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-43520-1_20

Mundt, M., Hong, Y., Pliushch, I., & Ramesh, V. (2023). A wholistic view of continual learning with deep neural networks: Forgotten lessons and the bridge to active and open world learning. *Neural Networks*, *160*, 306–336. https://doi.org/10.1016/j.neunet.2023.01.014

Nguyen, P., Tran, T., Le, K., Gupta, S., Rana, S., Nguyen, D., Nguyen, T., Ryan, S., & Venkatesh, S. (2021). Fast Conditional Network Compression Using Bayesian HyperNetworks. In N. Oliver, F. Pérez-Cruz, S. Kramer, J. Read, & J. A. Lozano (Eds.), *Machine Learning and Knowledge Discovery in Databases. Research Track* (pp. 330–345). Springer International Publishing. https://doi.org/10.1007/978-3-030-86523-8_20

Pandit, A., & Garg, A. (2021). Artificial Neural Networks in Healthcare: A Systematic Review. *2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence)*, 1–6. https://doi.org/10.1109/Confluence51648.2021.9377086

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., ... Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, *32*. Retrieved March 8, 2024, from https://proceedings.neurips.cc/paper_files/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

Pechey, R., & Monsivais, P. (2015). Supermarket Choice, Shopping Behavior, Socioeconomic Status, and Food Purchases. *American Journal of Preventive Medicine*, *49*(6), 868–877. https://doi.org/10.1016/j.amepre.2015.04.020

Rakesh, V., & Jain, S. (2021). Efficacy of Bayesian Neural Networks in Active Learning, 2601–2609. https://doi.org/10.1109/CVPRW53098.2021.00294

Ray, K., Szabó, B., & Clara, G. (2020). Spike and slab variational Bayes for high dimensional logistic regression. *Advances in Neural Information Processing Systems*, *33*, 14423–14434. Retrieved May 20, 2024, from https://proceedings.neurips.cc/paper_files/paper/2020/file/a5bad363fc47f424ddf5091c8471480a-Paper.pdf

Ren, P., Xiao, Y., Chang, X., Huang, P.-Y., Li, Z., Gupta, B. B., Chen, X., & Wang, X. (2021). A Survey of Deep Active Learning. *ACM Computing Surveys*, *54*(9), 180:1–180:40. https://doi.org/10.1145/3472291

Ricciuto, L., Tarasuk, V., & Yatchew, A. (2006). Socio-demographic influences on food purchasing among Canadian households [Publisher: Nature Publishing Group]. *European Journal of Clinical Nutrition*, *60*(6), 778–790. https://doi.org/10.1038/sj.ejcn.1602382

Ryu, S., Kwon, Y., & Kim, W. Y. (2019, March). Uncertainty quantification of molecular property prediction with Bayesian neural networks [arXiv:1903.08375 [cs, stat]]. https://doi.org/10.48550/arXiv.1903.08375

S. Eyke, N., H. Green, W., & F. Jensen, K. (2020). Iterative experimental design based on active machine learning reduces the experimental burden associated with reaction screening [Publisher: Royal Society of Chemistry]. *Reaction Chemistry & Engineering*, *5*(10), 1963–1972. https://doi.org/10.1039/D0RE00232A

Sagi, O., & Rokach, L. (2018). Ensemble learning: A survey [_eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1002/widm.1249]. *WIREs Data Mining and Knowledge Discovery*, *8*(4), e1249. https://doi.org/10.1002/widm.1249

Schubert, M., Riedlinger, T., Kahl, K., & Rottmann, M. (2023, September). Deep Active Learning with Noisy Oracle in Object Detection [arXiv:2310.00372 [cs]]. https://doi.org/10.48550/arXiv.2310.00372

Shannon, C. E. (1948). A mathematical theory of communication [Conference Name: The Bell System Technical Journal]. *The Bell System Technical Journal*, *27*(3), 379–423. https://doi.org/10.1002/j.1538-7305.1948.tb01338.x

Sharma, M., Rainforth, T., Teh, Y. W., & Fortuin, V. (2023, May). Incorporating Unlabelled Data into Bayesian Neural Networks [arXiv:2304.01762 [cs, stat]]. https://doi.org/10.48550/arXiv.2304.01762

Song, S., Berthelot, D., & Rostamizadeh, A. (2019, December). Combining MixMatch and Active Learning for Better Accuracy with Fewer Labels [arXiv:1912.00594 [cs, stat]]. https://doi.org/10.48550/arXiv.1912.00594

Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting [Publisher: JMLR. org]. *The journal of machine learning research*, *15*(1), 1929–1958. Retrieved March 28, 2024, from https://www.jmlr.org/papers/volume15/srivastava14a/srivastava14a.pdf?utm_content=buffer79b43&utm_medium=social&utm_source=twitter.com&utm_campaign=buffer,

Tegho, C. (2017). Bayes By Backprop Neural Networks for Dialogue Management. Retrieved May 15, 2024, from https://www.mlmi.eng.cam.ac.uk/files/tegho_dissertation.pdf

Tschacher, W., & Meier, D. (2020). Physiological synchrony in psychotherapy sessions [Publisher: Routledge _eprint: https://doi.org/10.1080/10503307.2019.1612114]. *Psychotherapy Research*, *30*(5), 558–573. https://doi.org/10.1080/10503307.2019.1612114

Turrell, G., & Kavanagh, A. M. (2006). Socio-economic pathways to diet: Modelling the association between socio-economic position and food purchasing behaviour. *Public Health Nutrition*, *9*(3), 375–383. https://doi.org/10.1079/PHN2005850

Uzair, M., & Jamil, N. (2020). Effects of Hidden Layers on the Efficiency of Neural networks [ISSN: 2049-3630]. *2020 IEEE 23rd International Multitopic Conference (INMIC)*, 1–6. https://doi.org/10.1109/INMIC50486.2020.9318195

Vabalas, A., Gowen, E., Poliakoff, E., & Casson, A. J. (2019). Machine learning algorithm validation with a limited sample size [Publisher: Public Library of Science]. *PLOS ONE*, *14*(11), e0224365. https://doi.org/10.1371/journal.pone.0224365

Wang, S., & Manning, C. (2013). Fast dropout training [ISSN: 1938-7228]. *Proceedings of the 30th International Conference on Machine Learning*, 118–126. Retrieved April 1, 2024, from https://proceedings.mlr.press/v28/wang13a.html

Wiltshire, T. J., Philipsen, J. S., Trasmundi, S. B., Jensen, T. W., & Steffensen, S. V. (2020). Interpersonal Coordination Dynamics in Psychotherapy: A Systematic Review. *Cognitive Therapy and Research*, *44*(4), 752–773. https://doi.org/10.1007/s10608-020-10106-3

Woo, J. O. (2022). Active Learning in Bayesian Neural Networks with Balanced Entropy Learning Principle. Retrieved May 22, 2024, from https://openreview.net/forum?id=ZTMuZ68B1g

Ying, X. (2019). An Overview of Overfitting and its Solutions [Publisher: IOP Publishing]. *Journal of Physics: Conference Series*, *1168*(2), 022022. https://doi.org/10.1088/1742-6596/1168/2/022022

Zeng, H., Yue, Z., Zhang, Y., Kou, Z., Shang, L., & Wang, D. (2022, October). On Attacking Out-Domain Uncertainty Estimation in Deep Neural Networks [arXiv:2210.02191 [cs]]. https://doi.org/10.48550/arXiv.2210.02191

Zhan, X., Qin, H., Liu, Y., Yao, L., Xie, W., Liu, G., & Zhou, J. (2020). Variational Bayesian Neural Network for Ensemble Flood Forecast-

ing [Number: 10 Publisher: Multidisciplinary Digital Publishing Institute]. *Water*, *12*(10), 2740. https://doi.org/10.3390/w12102740

Zhang, L., Carpenter, B., Gelman, A., & Vehtari, A. (2022, May). Pathfinder: Parallel quasi-Newton variational inference [arXiv:2108.03782 [cs, stat]]. https://doi.org/10.48550/arXiv.2108.03782

Zhang, X., Liang, F., Srinivasan, R., & Van Liew, M. (2009). Estimating uncertainty of streamflow simulation using Bayesian neural networks [_eprint: https://onlinelibrary.wiley.com/doi/pdf/10.1029/2008WR007030]. *Water Resources Research*, *45*(2). https://doi.org/10.1029/2008WR007030

Zhu, R., Chen, Y., Peng, W., & Ye, Z.-S. (2022). Bayesian deep-learning for RUL prediction: An active learning perspective. *Reliability Engineering & System Safety*, *228*, 108758. https://doi.org/10.1016/j.ress.2022.108758

6 Appendix1 Best-Case Models

6.1 Noisy-Data Classification Models Per Class

	Simple	LeNet-300-100	BBB
Accuracy	0.370229	0.384987	0.380582
F1	0.329115	0.363029	0.354872
Class 1 F1	0.552562	0.564390	0.545106
Class 2 F1	0.271760	0.235736	0.210826
Class 3 F1	0.065862	0.201005	0.268465
Class 4 F1	0.251862	0.312950	0.244489
Class 5 F1	0.503532	0.501064	0.505474
Class 1 Precision	0.667944	0.643678	0.680240
Class 2 Precision	0.282920	0.204694	0.168373
Class 3 Precision	0.037037	0.148148	0.233259
Class 4 Precision	0.240621	0.337646	0.204469
Class 5 Precision	0.626098	0.590966	0.642691
Class 1 Recall	0.471171	0.502493	0.454764
Class 2 Recall	0.261446	0.277876	0.281905
Class 3 Recall	0.297030	0.312500	0.316188
Class 4 Recall	0.264205	0.291620	0.303987
Class 5 Recall	0.421097	0.434903	0.416541
Achieved at epoch	95	90	83

Table 12: shows the per class evaluation metrics for the noisy data classification models. This is an extension of Table 3.

6.2 Relaxed-Data Classification Models Per Class

	Simple	LeNet-300-100	BBB
Accuracy	0.993333	0.993000	0.995333
F1	0.992999	0.992726	0.995272
Class 1 F1	0.997991	0.996656	0.995989
Class 2 F1	0.994764	0.993897	0.994755
Class 3 F1	0.990222	0.991087	0.993767
Class 4 F1	0.987319	0.987273	0.992727
Class 5 F1	0.994700	0.994718	0.999120
Class 1 Precision	1.000000	1.000000	1.000000
Class 2 Precision	0.991304	0.991304	0.989565
Class 3 Precision	0.994643	0.992857	0.996429
Class 4 Precision	0.989111	0.985481	0.990926
Class 5 Precision	0.989455	0.992970	0.998243
Class 1 Recall	0.995989	0.993333	0.992011
Class 2 Recall	0.998249	0.996503	1.000000
Class 3 Recall	0.985841	0.989324	0.991119
Class 4 Recall	0.985533	0.989071	0.994536
Class 5 Recall	1.000000	0.996473	1.000000
Achieved at epoch	72	84	45

Table 13: shows the per class evaluation metrics for the relaxed data classification models. This is an extension of Table 4.

6.3 Single Instance Models

In this part of the thesis a single instance of the trained model is tested. Because the closed form models use the scaled normal prior, inference can be achieved with a single feedforward pass, but achieving uncertainties for the model predictions necessitates passing unlabeled data through the model multiple times, or for multiple instances. Because this is computationally more expensive to implement than to realize a single instance of the model, and because this thesis, attempts to answer what the advantages of BNNs are, a single instance of this model makes sense to include in the thesis. Apart from being utilized for uncertainty predictions, BNNs are also widely praised for their immunity to overfitting. Demonstrating a single realization of the closed form model, means that the models become theoretically very similar to regular dropout neural networks, but that they should still be able to show their resistance to overfitting. In this part of this thesis, a single instance is tested for all the three implemented models for both the regression

and classification datasets.

	Simple	LeNet-300-100	BBB
RMSE (validation)	0.839	0.888	0.699
Upper Confidence	0.55	0.515625	0.53125
Lower Confidence	0.45	0.484375	0.46875

Table 14: shows results for the single instance regression models run on the noisy data

The results of the trained models show that the Bayes by Backprop achieved the best RMSE on the heldout validation dataset. Comparing it to the models that used the closed form implementation provided by Molchanov et al., (2017), it is clearly achieving the lowest RMSE score with a score of 0.699 compared to 0.839 and 0.888 for the simple and dense models respectively, see table 14 for reported model results. However, when looking at the loss curves of the model training process, it becomes clear that the Bayes by Backprop model is showing clear signs of overfitting with a training loss curve that is far lower than the loss curves for testing and validation, see 14. Relating this observation to the fact that the RMSE score for that exact same model was obtained for the heldout validation dataset, it becomes ambigous to interpret why the model's learning process seems to be reflective of overfitting, while the model itself performs quite well on never before seen data. The simple closed form model shows a classic converging loss curve pattern with training and test curves converging towards a common value after having decreased with the number of times it has seen the training data. The dense closed form model shows a non-converging pattern, but learning was halted early due to no significant rise in model performance when taking into account RMSE.

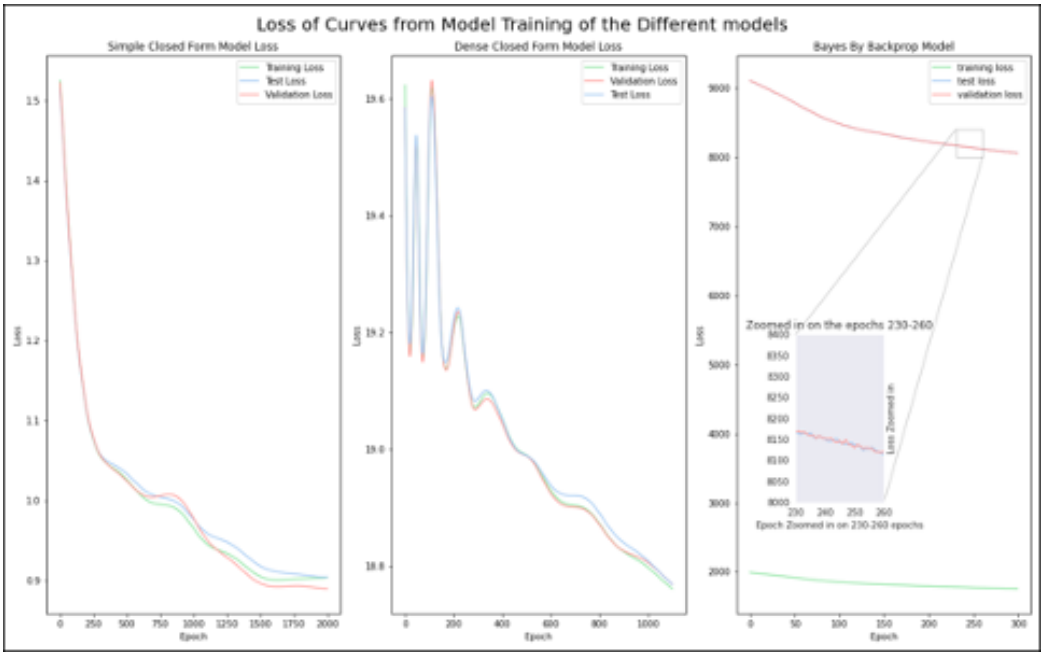


Figure 14: loss curves from the training process of the different models.

Figure 14, shows the RMSE curves for the 3 regression models. From the illustration it is obvious that the simple model achieves its best RMSE early and then flatlines at around 1500 epochs. The dense model has a harder time stabilizing but shows somewhat decent stability at around the same value as the simple model with an RMSE score just below 0.84. Notably, it achieves its best RMSE-score by training on much fewer epochs than the simple model. The Bayes by Backprop model shows that the RMSE for the training data fluctuates a lot during training but achieves very good RMSE on the test and validation data.

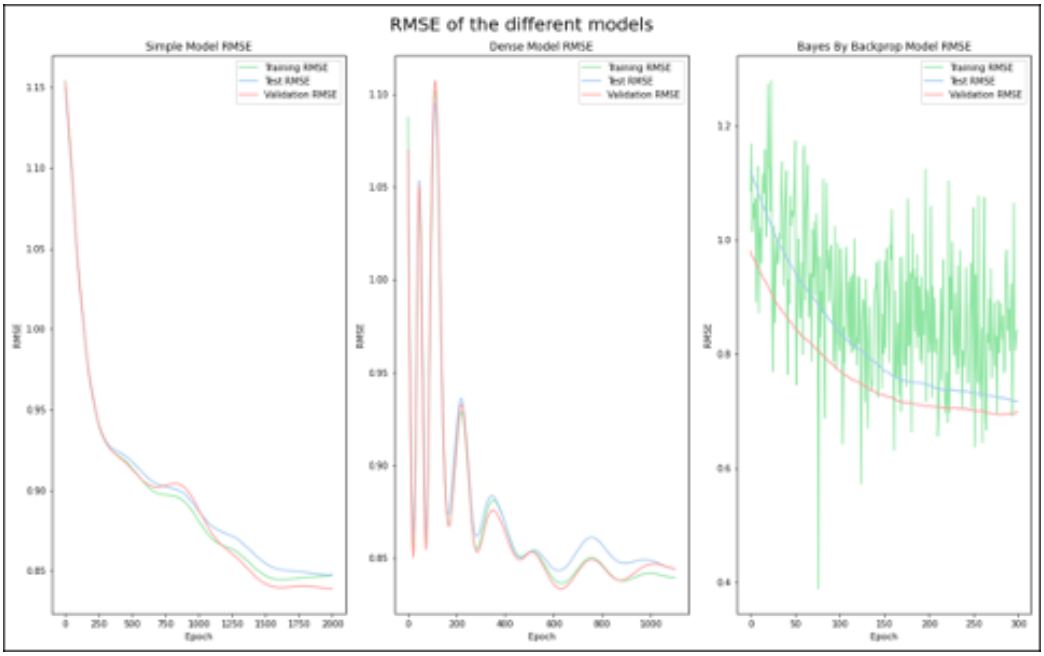


Figure 15: RMSE curves from the training process of the different models.

Figure 16 shows the models' predicted values compared to the true values. The x-axes show the target variables scaled. Mean values are used for the prediction curve. All the three models show very similar predictive performance on the validation data. All of them seem to find the general pattern, but are too narrow in their predictions, i.e., all the predictions are within the most commonly occurring range of values. Overall, the models perform decently and seem to have learned the general pattern of the data. Keeping in mind, how the data was simulated e.g., with lots of random noise and a variable such as "guest" that potentially introduced conflicting variability in the outcome variables, the models are all deemed solid.

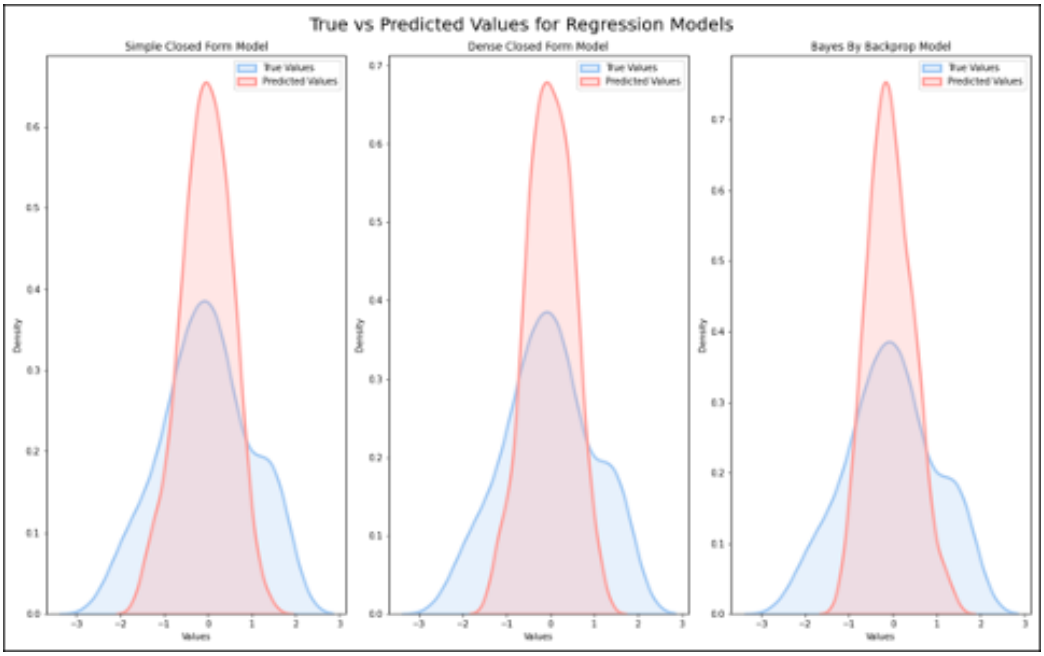


Figure 16: True values from the validation data set in blue and the models equivalently predicted values in pink.

All in all, three different BNN models were trained for a regression task on a simulated dataset. Overall performance showed very similar results for all the models with the Bayes By Backprop model performing with the best RMSE score 0.699, compared to 0.839 for the simple closed form model and 0.888 for the dense closed form model. However, the better performance should be seen in the light of a poorly converging model that shows some signs of overfitting to the training data, and some signs of not learning the patterns of the training data completely.

	Simple Closed Form	Large Closed Form	Bayes by Backprop
Accuracy	33.4	32.4	30.0666

Table 15: shows the results for the trained classification models.

The classification models all achieved very similar accuracy-scores. However, the simple model achieved the highest with a score 33.4 compared to 32.4 and 30.0666 for the large, closed form model and the Bayes by Backprop model. Looking at the loss curves for the models in 17, it looks like none of them converges perfectly, however the closed form models both show decreasing trends for training-, testing-, and validation-curves. It is possible that training them for additional epochs would see them converge with stabilizing loss curves,

however, training was intentionally cut short due to non-increasing accuracy scores. The Bayes by Backprop model shows problematic learning curves with almost stationary loss curves for training-, test-, and validation-loss. However, as was the case for its regression counterpart it achieves similar accuracy scores as the other models.

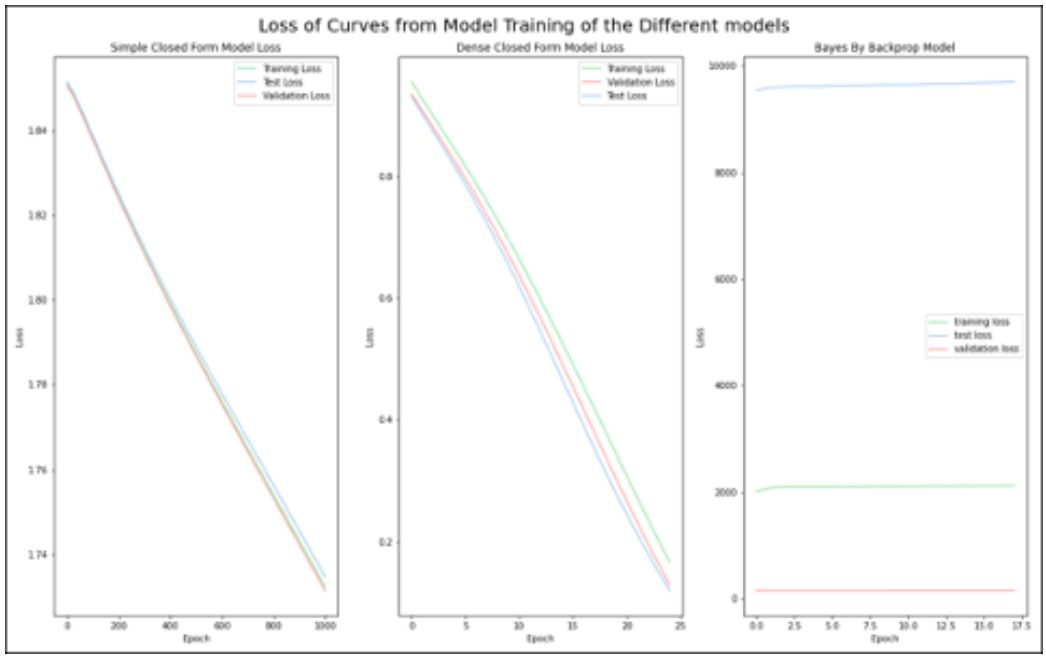


Figure 17: shows loss curves for the training process of the classification models.

The accuracy plots in 18, show that the simple, closed form model has the most ideal accuracy curve for its learning process. The pattern is abruptly increasing in the beginning until it starts to become more steadily increasing to the point where it becomes almost stationary. The large, closed form model shows a decent pattern, but is not finding a steady state to the same degree that the simple model is. Training has been cut short for the large model, because the accuracy started to decrease before it found a stabilizing place. The Bayes by Backprop model shows a similar pattern to the large, closed form model with accuracy curves that never completely find a resting pace before training has been cut short.

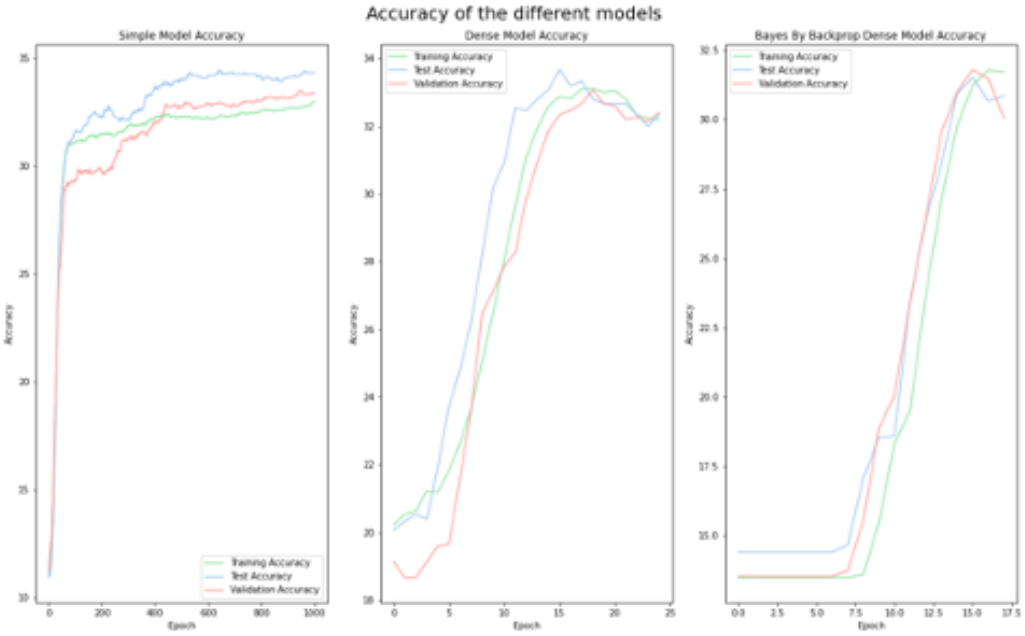


Figure 18: shows accuracy curves for the training process of the classification models.

In conclusion, the simple regression model was the best performing model both in terms of good converging and accuracy score. The large, closed model performed second best, but showed signs of not fitting the training data properly. The Bayes by Backprop model showed poor learning.

7 Appendix2 Continual- and Active Learning Models

7.1 Continual Learning Classification Models Per Class

Table 16: Model Performance Metrics

	Simple	LeNet-300-100
Accuracy	0.94037	0.93037
F1	0.934347	0.923526
Precision	0.935534	0.924609
Recall	0.939959	0.934682
Class 1 F1	0.989278	0.986553
Class 2 F1	0.953008	0.938
Class 3 F1	0.90566	0.889096
Class 4 F1	0.871676	0.850711
Class 5 F1	0.952113	0.953271
Class 1 Precision	0.98156	0.988652
Class 2 Precision	0.984466	0.91068
Class 3 Precision	0.940206	0.983505
Class 4 Precision	0.77732	0.740206
Class 5 Precision	0.994118	1
Class 1 Recall	0.997118	0.984463
Class 2 Recall	0.923497	0.96701
Class 3 Recall	0.873563	0.811224
Class 4 Recall	0.992105	1
Class 5 Recall	0.913514	0.910714
Achieved in round	10	1

Table 17: Model Performance Metrics per class for the continually trained regression models trained on the relaxed data for 10 rounds

7.2 Active Learning Regression Models

These models were trained on a dataset not presented in the thesis. However, the data was very resembling of the noisy data with no attempts to try and even out the output space. In the main parts of the thesis, the datasets have been relatively balanced, whereas the models presented here were trained on imbalanced datasets. The models presented here are *actively* trained,

i.e., not continually as the models in the main chapters of the thesis. They can therefore not be used for comparison. However, they should be able to provide some insights into what happens when more training data is available for training, seeing as every single round is instantiated the same. It is obvious that the closed form models are way better at resisting the noise induced by the oracle, adn instead learn from the general patterns being provided by the growing training dataset. On the other hand, the BBB model seems to overfit on the spurious correlations introduced by the noise and starts to drop in performance immediately.

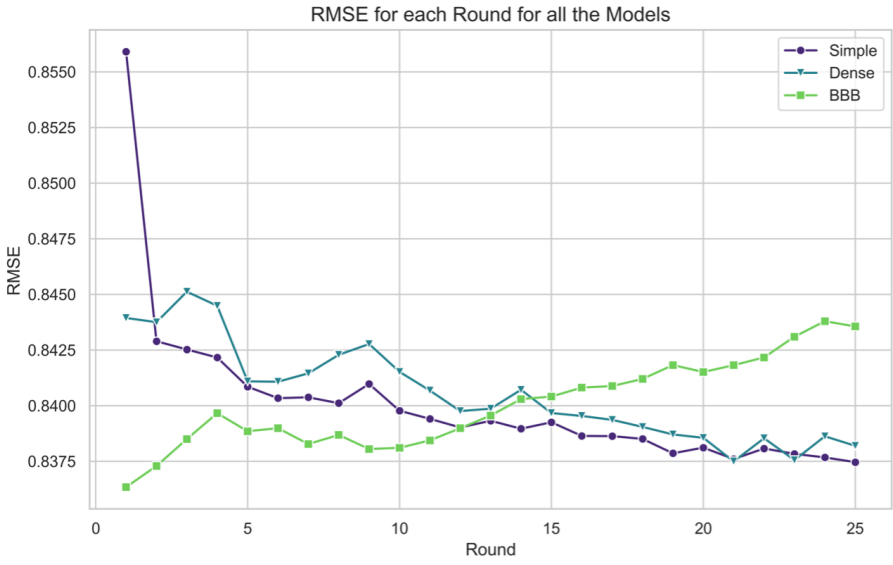


Figure 19: RMSE for all the regression models per round of active learning for the noisy imbalanced data.

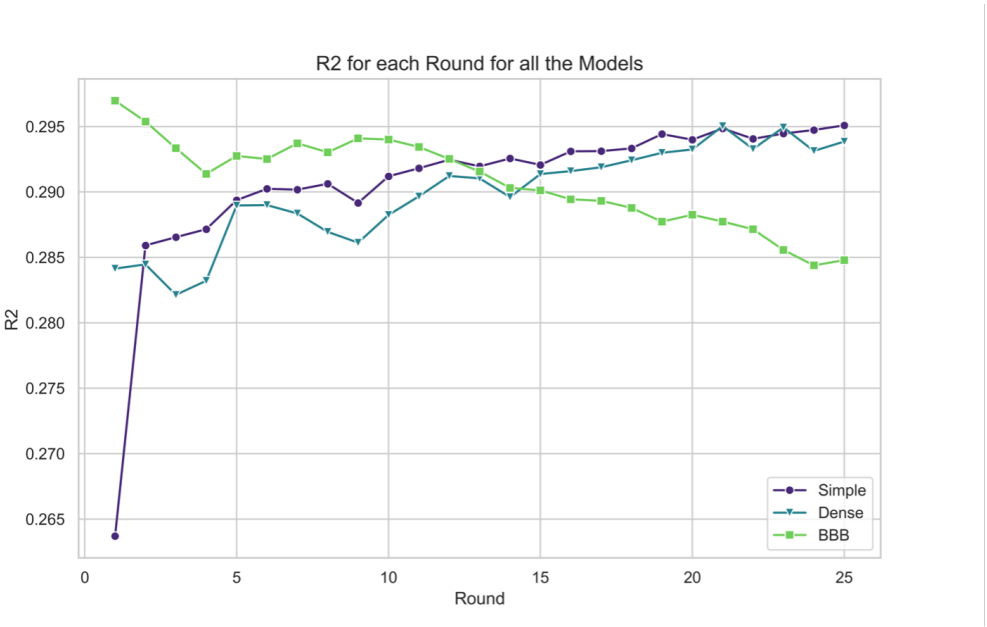


Figure 20: R^2 for all the regression models per round of active learning for the noisy imbalanced data.

	R^2	MSE	RMSE	Adjusted R^2	Achieved in round
Simple	0.295091	0.701345	0.837463	0.294045	25
Dense	0.295037	0.701399	0.837495	0.293991	21
BBB	**0.296979**	**0.699467**	**0.836341**	**0.295935**	**1**

Table 18: Regression Actively Trained Model Performance Metrics on the imbalanced noisy dataset. Best metrics are highlighted in bold.

7.3 Active Learning Classification Models

The models presented here are continually trained for 25 rounds on noisy imbalanced data.

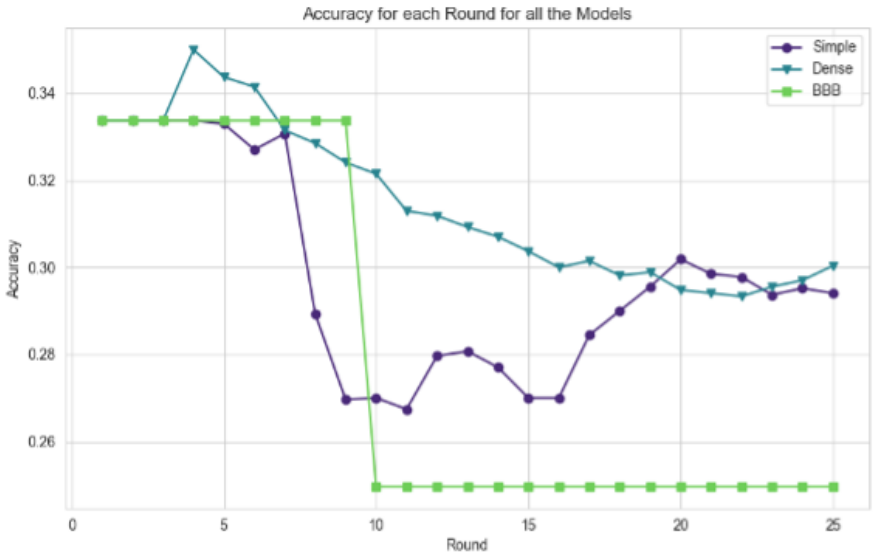


Figure 21: Accuracy for all the classification models per round of active learning for the noisy imbalanced data.

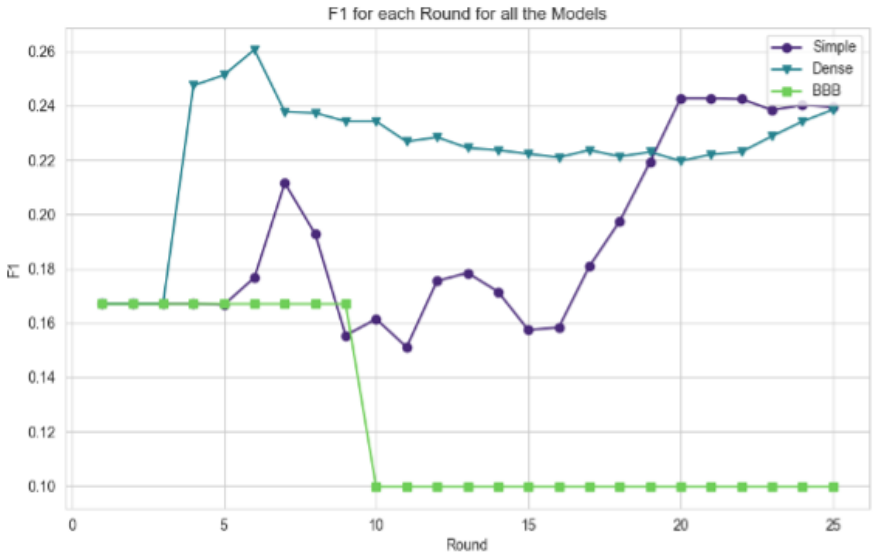


Figure 22: F1 for all the classification models per round of active learning for the noisy imbalanced data.

8 Appendix3 Dyadic Synchrony

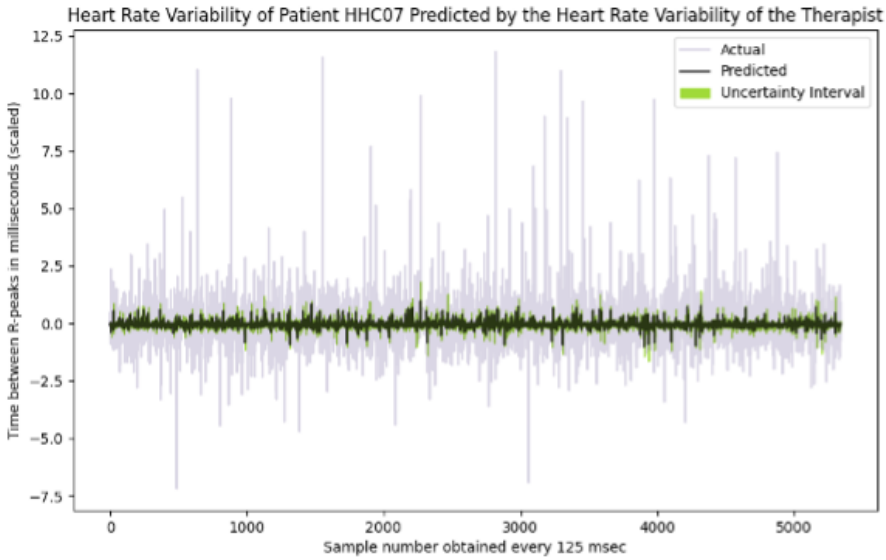


Figure 23: Shows the HRV of the patient and the predicted HRV of the patient. uncertainty estimates are included in green

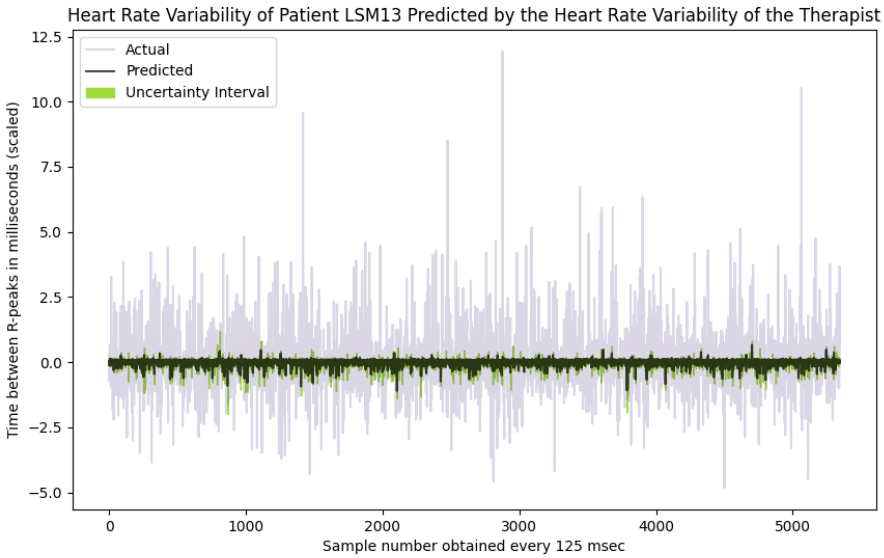


Figure 24: Shows the HRV of the patient and the predicted HRV of the patient. uncertainty estimates are included in green